



Gymnázium J. K. Tyla

DarkFactory

Agentické a harnessové inženýrství:

Umělá inteligence v praxi

ODBORNÁ PRÁCE

Autor práce: Patrik Marius, 4.D
Vedoucí práce: Michal Dočekal

Rozsah práce: 8646 slov / 58082 znaků

Prohlášení

Prohlašuji, že jsem tuto studentskou odbornou práci vypracoval/a samostatně pod dohledem vedoucího uvedeného na první straně. Všechny použité zdroje jsou uvedeny v seznamu zdrojů a informace z nich získané jsou v textu řádně označeny odkazem na zdroj. Souhlasím s tím, aby tištěná forma práce byla uchována na Gymnázium J. K. Tyla a tam používána jako tištěný zdroj např. pro další studentské práce či pro prezentaci vzdělávání na GJKT.

V Hradci Králové dne

Podpis autora práce:

.....

Anotace

Tato odborná práce se zabývá současným využitím agentní umělé inteligence při vývoji softwaru. Popisuje, jaké části vývojového procesu mohou dnešní agentní systémy provádět samostatně a jaké nástroje, pravidla a kontrolní mechanismy potřebují pro opakovatelnou práci nad skutečným repozitářem. Praktickou část tvoří návrh a implementace systému DarkFactory, který tyto principy převádí do konkrétní architektury agentního harnessu.

Klíčová slova

Actions (GitHub Actions) , Agent , Agent Divergence Pathologies (Patologie divergence) , Agent Harness (Agentní harness) , Agent Loop (Smyčka ReAct) [ReAct Loop] , Agent Session (Agentní sezení) , Agentic Engineering (Agentické inženýrství) , Autoregressive Modeling (Autoregresivní modelování) , Branch (Větev repozitáře) [Repository Branch] , Branch Protection (Ochrana větví) , Chatbot , ChatGPT , ChatGPT App Interface (Rozhraní aplikace ChatGPT) , CI (Průběžná integrace) [Continuous Integration] , Claude Code , Claude Code Interface (Rozhraní Claude Code) , Claude Desktop (Aplikace Claude pro desktop) [Claude Desktop App] , Claude Desktop Interface (Rozhraní aplikace Claude) , Claude Opus 5 , Claude Opus 5 Product Visual (Produktový vizuál Claude Opus 5) , Code Execution (Spouštění kódu) , Codex , Codex App Interface (Rozhraní aplikace Codex) , Compaction (Kompakce kontextu) [Context Compaction] , Conclusion (Závěr) , Container (Softwarový kontejner) [Software Container] , Context Engineering (Kontextové inženýrství) , Context Rot (Degradace kontextu) , Context Window (Kontextové okno) , DAG (Orientovaný acyklický graf) [Directed Acyclic Graph] , DeepSeek-V4.1-Flash , DeepSeek-V4.1-Flash Product Visual (Produktový vizuál DeepSeek-V4.1-Flash) , Development Environment and Practices (Vývojové prostředí a praxe) , Embedding (Vektorová reprezentace) , Evaluation Limitations (Omezení evaluace) , Evaluation Snapshot (Evaluační snapshot) , Flaky Test (Nestálý test) , Git , GitHub , Global Generative AI Adoption (Globální adopce generativní AI) , GPT-5.6 , GPT-5.6 Product Visual (Produktový vizuál GPT-5.6) , Graph Engineering (Inženýrství pracovních grafů) [Workflow-graph Engineering] , Guardrail (Deterministický mantinel) [Deterministic Guardrail] , HITL (Zapojení člověka do smyčky) [Human-in-the-loop] , Hook (Událostní zachytný bod) [Event Hook] , Introduction (Úvod) , Issue (Úloha GitHubu) [GitHub Issue] , JSON Schema Tool Calling (Vyvolávání nástrojů s JSON Schema) , KV Cache (Mezipaměť klíčů a hodnot) [Key–Value Cache] , Language Models, Chatbots, and Agents (Jazykové modely, chatboti a agenti) , LLM (Velký jazykový model) [Large Language Model] , Loop Engineering (Inženýrství prováděcí smyčky) [Execution-loop Engineering] , Main Goal (Hlavní cíl) , MCP (Model Context Protocol) , Merge (Sloučení větví) [Branch Merge] , Methodology (Metodika práce) , Motivation and Problem Definition (Motivace a vymezení problému) , Planning (Plánování) , Plugins (Rozšíření) , Progressive Disclosure (Postupné zpřístupňování kontextu) , Prompt Engineering (Promptové inženýrství) , Pull Request (Požadavek na sloučení) , RAG (Generování rozšířené vyhledávání) [Retrieval-Augmented Generation] , ReAct Agent-loop Diagram (Schéma agentní smyčky ReAct) , Required Checks (Požadované kontroly) , Research Question Evaluation (Vyhodnocení výzkumných otázek) , Research Questions (Výzkumné otázky) , Results and Discussion (Výsledky a diskuse) , Runtime (Běhové prostředí) [Runtime Environment] , Sandbox (Izolované běhové prostředí) , Script (Skript) , Semantic Drift (Sémantický posun) , SKILL.md (Formát SKILL.md) [SKILL.md Format] , Skills (Dovednosti) , Slop , Software Engineering (Softwarové inženýrství) , Squash (Sloučení commitů) [Commit Squashing] , Stochastic Decoding (Stochastické dekódování) , Sub-goals (Dílčí cíle) , Subagent (Podřízený agent) , System Prompt (Systémový prompt) , Thesis Objective and Research Questions (Cíl práce a výzkumné otázky) , Token , Tokenizer (Tokenizér) , Tool Calling (Vyvolávání nástrojů) , Transformer (Transformerová architektura) [Transformer Architecture] , Turn (Tah interakce) [Interaction Turn] , Verified Implementation State (Ověřený stav implementace) , Version control (Správa verzí) , Vibe Coding (Vibe coding)

Obsah

Prohlášení	2
Anotace	3
Klíčová slova	4
1 Introduction (Úvod)	9
1.1 Motivation and Problem Definition (Motivace a vymezení problému)	9
1.1.1 Global Generative AI Adoption (Globální adopce generativní AI)	9
1.2 Thesis Objective and Research Questions (Cíl práce a výzkumné otázky)	10
1.2.1 Main Goal (Hlavní cíl)	10
1.2.2 Sub-goals (Dílčí cíle)	10
1.2.3 Research Questions (Výzkumné otázky)	10
1.3 Methodology (Metodika práce)	11
2 Development Environment and Practices (Vývojové prostředí a praxe)	12
2.1 GitHub	12
2.2 Runtime (Běhové prostředí) [Runtime Environment]	12
2.3 Software Engineering (Softwarové inženýrství)	12
2.3.1 Planning (Plánování)	13
2.3.2 DAG (Orientovaný acyklický graf) [Directed Acyclic Graph]	13
2.3.3 Version control (Správa verzí)	13
2.3.3.1 Git	13
2.3.3.1.1 Branch (Větev repozitáře) [Repository Branch]	14
2.3.3.1.2 Merge (Sloučení větví) [Branch Merge]	14
2.3.3.1.3 Squash (Sloučení commitů) [Commit Squashing]	14
2.3.3.1.4 Issue (Úloha GitHubu) [GitHub Issue]	15
2.3.3.1.5 Pull Request (Požadavek na sloučení)	15
2.3.3.1.6 Required Checks (Požadované kontroly)	15
2.3.3.1.7 Branch Protection (Ochrana větví)	15
2.4 CI (Průběžná integrace) [Continuous Integration]	16
2.4.1 Actions (GitHub Actions)	16
2.4.2 Container (Softwarový kontejner) [Software Container]	16
2.4.3 Flaky Test (Nestálý test)	16
3 Language Models, Chatbots, and Agents (Jazykové modely, chatboti a agenti)	18
3.1 Chatbot	18
3.1.1 ChatGPT	18
3.1.1.1 ChatGPT App Interface (Rozhraní aplikace ChatGPT)	18

3.2	Agent	19
3.3	LLM (Velký jazykový model) [Large Language Model]	19
3.3.1	GPT-5.6	20
3.3.1.1	GPT-5.6 Product Visual (Produktový vizuál GPT-5.6)	20
3.3.2	Claude Opus 5	20
3.3.2.1	Claude Opus 5 Product Visual (Produktový vizuál Claude Opus 5)	20
3.3.3	DeepSeek-V4.1-Flash	21
3.3.3.1	DeepSeek-V4.1-Flash Product Visual (Produktový vizuál DeepSeek-V4.1-Flash)	21
3.3.4	Transformer (Transformerová architektura) [Transformer Architecture]	22
3.3.5	Autoregressive Modeling (Autoregresivní modelování)	22
3.3.6	Stochastic Decoding (Stochastické dekódování)	22
3.3.7	Tokenizer (Tokenizér)	22
3.3.8	Token	22
3.3.9	Embedding (Vektorová reprezentace)	23
3.3.10	Context Window (Kontextové okno)	23
3.3.11	KV Cache (Mezipaměť klíčů a hodnot) [Key-Value Cache]	24
3.3.12	Turn (Tah interakce) [Interaction Turn]	24
4	Agentic Engineering (Agentické inženýrství)	25
4.1	Vibe Coding (Vibe coding)	25
4.2	Slop	25
4.3	Prompt Engineering (Promptové inženýrství)	25
4.3.1	System Prompt (Systémový prompt)	26
4.4	Agent Harness (Agentní harness)	26
4.4.1	Codex	26
4.4.1.1	Codex App Interface (Rozhraní aplikace Codex)	26
4.4.2	Claude Code	27
4.4.2.1	Claude Code Interface (Rozhraní Claude Code)	27
4.4.3	Claude Desktop (Aplikace Claude pro desktop) [Claude Desktop App] . . .	28
4.4.3.1	Claude Desktop Interface (Rozhraní aplikace Claude)	28
4.4.4	Sandbox (Izolované běhové prostředí)	29
4.4.5	Guardrail (Deterministický mantinel) [Deterministic Guardrail]	30
4.4.6	HITL (Zapojení člověka do smyčky) [Human-in-the-loop]	30
4.4.7	Plugins (Rozšíření)	30

4.4.8	Agent Loop (Smyčka ReAct) [ReAct Loop]	31
4.4.8.1	ReAct Agent-loop Diagram (Schéma agentní smyčky ReAct)	31
4.4.8.2	Agent Session (Agentní sezení)	32
4.4.8.3	Loop Engineering (Inženýrství prováděcí smyčky) [Execution-loop Engineering]	32
4.4.8.4	Agent Divergence Pathologies (Patologie divergence)	33
4.4.9	Tool Calling (Vyvolávání nástrojů)	33
4.4.9.1	JSON Schema Tool Calling (Vyvolávání nástrojů s JSON Schema)	33
4.4.9.2	Code Execution (Spouštění kódu)	33
4.4.9.3	MCP (Model Context Protocol)	34
4.4.10	Skills (Dovednosti)	34
4.4.10.1	SKILL.md (Formát SKILL.md) [SKILL.md Format]	34
4.4.10.2	Progressive Disclosure (Postupné zpřístupňování kontextu)	35
4.4.10.3	Script (Skript)	36
4.4.10.4	Hook (Událostní zachytný bod) [Event Hook]	36
4.4.11	Context Engineering (Kontextové inženýrství)	36
4.4.11.1	Compaction (Kompakce kontextu) [Context Compaction]	36
4.4.11.2	RAG (Generování rozšířené vyhledáváním) [Retrieval-Augmented Generation]	37
4.4.11.3	Context Rot (Degradace kontextu)	37
4.4.11.4	Semantic Drift (Sémantický posun)	38
4.4.12	Graph Engineering (Inženýrství pracovních grafů) [Workflow-graph Engineering]	38
4.4.12.1	Subagent (Podřízený agent)	39
5	Results and Discussion (Výsledky a diskuse)	40
5.1	Evaluation Snapshot (Evaluační snapshot)	40
5.2	Verified Implementation State (Ověřený stav implementace)	40
5.3	Research Question Evaluation (Vyhodnocení výzkumných otázek)	41
5.4	Evaluation Limitations (Omezení evaluace)	41
6	Conclusion (Závěr)	43
	Seznam obrázků a tabulek	44
	Seznam příloh	45
	Seznam zdrojů	46

1 Introduction (Úvod)

Tato práce zkoumá, jak dnes účinně využívat agentní umělou inteligenci při vývoji softwaru. Zaměřuje se na to, co současní agenti dokážou samostatně provést a jaké prostředí, nástroje a pravidla potřebují, aby jejich práce byla opakovatelná a kontrolovatelná. Praktickým příkladem je DarkFactory, na kterém jsou popsány principy převedeny do konkrétní architektury a vývojového procesu. Cílem úvodu je stručně vymezit, co práce zkoumá, proč je téma aktuální a podle čeho bude navržené řešení posuzováno.

1.1 Motivation and Problem Definition (Motivace a vymezení problému)

Motivací práce je ukázat, jaké úlohy dokážou současné agentní systémy samostatně provádět a jaká technická vrstva umožňuje převést schopnost jazykového modelu do spolehlivého jednání nad skutečným softwarovým projektem. Generativní AI se během několika let rozšířila do masového používání. Microsoft AI Economy Institute odhaduje, že v prvním čtvrtletí 2026 použilo generativní AI 17,8 % světové populace v produktivním věku. Současně se rozšiřují systémy, které plánují více kroků, pracují se soubory a nástroji, spouštějí příkazy a testy a vykonávají delší úlohy.⁽¹⁾ V softwarovém inženýrství tento posun reprezentují například Codex a Claude Code. Jejich oficiální popisy uvádějí práci nad skutečnými repozitáři, úpravy souborů, spouštění testů a další činnosti přesahující jednorázové generování kódu.^(2, 3) Práce proto sleduje praktické využití těchto systémů při vývoji softwaru: jak jim zpřístupnit repozitář, nástroje a ověřování tak, aby dokázaly samostatně provádět užitečnou část vývojové práce a člověk si zachoval kontrolu nad důležitými rozhodnutími.

1.1.1 Global Generative AI Adoption (Globální adopce generativní AI)

Grafický podklad dokumentuje rychlost, s níž se generativní AI rozšířila z experimentální technologie do běžného používání. Microsoft AI Economy Institute uvádí pro druhé pololetí roku 2025 celosvětový podíl uživatelů 16,3 %, oproti 15,1 % v prvním pololetí⁽⁴⁾. Novější zpráva pro první čtvrtletí 2026 odhaduje používání generativní AI na 17,8 % světové populace v produktivním věku⁽¹⁾.

Region	H1 2025 AI Diffusion	H2 2025 AI Diffusion	Change
Global North	22.9%	24.7%	+1.8
Global South	13.1%	14.1%	+1.0
World	15.1%	16.3%	+1.2

Obrázek 1: AI diffusion v globálním Severu, globálním Jihu a celosvětově, H1–H2 2025. Zdroj: Microsoft AI Economy Institute⁽⁴⁾.

Měřítko adopce neposuzuje schopnosti jednotlivých systémů, ale dokládá jejich rychlé rozšíření do reálného používání. Výchozím problémem je převést schopnosti současných modelů do opakovatelného a kontrolovatelného agentního vývojového procesu.

1.2 Thesis Objective and Research Questions (Cíl práce a výzkumné otázky)

Cíl práce a výzkumné otázky převádějí motivaci do konkrétního návrhového cíle, dílčích úkolů a otázek, podle nichž lze výsledný systém posoudit.

Hlavní cíl určuje výsledný předmět návrhu. Dílčí cíle rozkládají práci na nezbytné oblasti a výzkumné otázky formulují problémy, které musí architektura a její evaluace zodpovědět. Tato struktura propojuje motivaci s metodikou a výsledky: každá část práce má být dohledatelná k některému cíli nebo výzkumné otázce.

1.2.1 Main Goal (Hlavní cíl)

Zjistit, jak lze současnou agentní AI účinně používat při vývoji softwaru, a navrhnout architekturu agent harnessu, která podporuje vysokou míru autonomie při zachování lidského dohledu v důležitých rozhodnutích.

Cíl spojuje popis současných agentních postupů s návrhem konkrétního systému Dark-Factory. Výsledná architektura má podporovat samostatné provádění vývojových úloh a současně zachovat jasná místa pro lidskou kontrolu. Výsledkem práce má být použitelný návrh agentního harnessu a popis principů, které umožňují současnou agentní AI efektivně zapojit do vývoje softwaru.

1.2.2 Sub-goals (Dílčí cíle)

- Vymezit infrastrukturu pro správu verzí a průběžné automatické ověřování změn.
- Popsat limity modelového kontextu, jeho správu a mechanismy pro dlouhotrvající agentní úlohy.
- Popsat nástrojové a prováděcí mechanismy současného agentního harnessu.
- Navrhnout způsob, jak zachovat lidský dohled nad důležitými rozhodnutími bez nutnosti ručně provádět každou rutinní změnu.

Dílčí cíle rozkládají hlavní cíl na vývojové prostředí, práci s modelem a kontextem, řízení agentního běhu a lidskou kontrolu. Každá oblast je dále rozpracována jako samostatné koncepty. Splnění dílčích cílů vytváří podklady pro návrh DarkFactory a pro následné zodpovězení výzkumných otázek.

1.2.3 Research Questions (Výzkumné otázky)

- VO1: Jak lze současný agentní systém zapojit do vývoje softwaru tak, aby samostatně prováděl co největší část běžné práce a člověk se soustředil na zadání, kontrolu a důležitá rozhodnutí?
- VO2: Jaké mechanismy harnessu pomáhají rozpoznat a zastavit neproduktivní opakování, oscilaci nebo zacyklení během delší úlohy?

- VO3: Jak spravovat pracovní kontext agenta tak, aby při delších úlohách neztrácel důležité požadavky a stav projektu?

Otázky soustřeďují hodnocení na praktickou míru samostatnosti, spolehlivost prováděcí smyčky a zachování potřebného pracovního kontextu. Výsledky práce se mají vztahovat k tomu, jak dobře navržená architektura podporuje samostatnou práci agenta, řízení problémového běhu a zachování správného kontextu.

1.3 Methodology (Metodika práce)

Práce porovnává veřejně popsané principy současných agentních systémů, rozděluje je do samostatných konceptů a používá je při návrhu DarkFactory.

Předmětem práce není trénování neuronových sítí, optimalizace vah ani podrobná matematika modelového učení. **LLM (Velký jazykový model) [Large Language Model]** je chápán jako hotová inferenční komponenta a je popsán pouze v rozsahu potřebném pro další části práce.

Architektura kolem modelu je rozložena do samostatných konceptů, aby měl každý mechanismus vlastní definici, popis a shrnutí a nebylo nutné stejné vysvětlení opakovat v několika kapitolách.

Navržené principy jsou následně promítnuty do DarkFactory. Vlastnosti systému se mají posuzovat podle ověřitelných výstupů vývojového procesu, nikoli podle předem předpokládaných výsledků. Metodika postupuje od veřejných zdrojů a vymezení konceptů přes návrh konkrétní architektury až k ověření jejích vlastností na skutečných výstupech systému.

2 Development Environment and Practices (Vývojové prostředí a praxe)

Vývojové prostředí a praxe tvoří soubor verzovacích, plánovacích, integračních a kontrolních postupů, které poskytují deterministický rámec pro agentní vývoj softwaru. Agentní systém nepracuje pouze s textem, ale se stavem repozitáře a vývojovým procesem. Proto musí být jeho akce ukotveny v explicitních postupech pro verzování, plánování změn, automatické kontroly a revizi výsledků. Deterministické vývojové prostředí převádí jednotlivé agentní akce na dohledatelný a ověřitelný softwarově-inženýrský proces.

2.1 GitHub

GitHub je cloudová platforma pro hosting gitových repozitářů a koordinaci vývojového cyklu pomocí Issues, Pull Requests a automatizačních workflow. GitHub poskytuje nad gitovým repozitářem koordinační vrstvu pro zadání práce, revizi změn a automatizaci. Tyto odpovědnosti jsou v práci dále rozloženy mezi samostatné koncepty Issue, Pull Request a GitHub Actions. GitHub zde tvoří koordinační vrstvu nad Gitem, která propojuje zadání, revizi změn a automatizované kontroly.

2.2 Runtime (Běhové prostředí) [Runtime Environment]

Běhové prostředí je prostředí, ve kterém se program nebo agent skutečně vykonává a ve kterém má k dispozici procesy, souborový systém, proměnné prostředí, síť a další systémové prostředky. Pro agentní systém určuje runtime praktické hranice toho, co může během úlohy provést. Harness může běh izolovat v kontejneru nebo jiném sandboxu, přidělit mu pracovní adresář, potřebné nástroje a oprávnění a po dokončení zachovat pouze požadované výstupy. Oddělení runtime od modelu je důležité také pro reprodukovatelnost: stejný model může v různých prostředích disponovat zcela odlišnými nástroji, soubory a systémovými možnostmi. Runtime převádí požadavek agenta na skutečně proveditelné operace a současně vymezuje technické hranice jejich provádění.

2.3 Software Engineering (Softwarové inženýrství)

Softwarové inženýrství je systematické uplatňování inženýrských principů na specifikaci, návrh, implementaci, ověřování, provoz a údržbu softwarových systémů. Pro agentní systémy je tento rámec důležitý proto, že generování kódu představuje pouze jednu část životního cyklu změny. Výsledek musí být zasazen do specifikace požadavku, řízeného procesu změn, automatického ověřování a následné revize. Agentní vývoj je podmnožinou softwarového inženýrství: kvalita výsledku se neposuzuje podle množství vygenerovaného kódu, ale podle splnění požadavků a ověřitelnosti změny.

2.3.1 Planning (Plánování)

Plánování je proces převodu požadavku na explicitní posloupnost kroků, závislostí a ověřovacích podmínek před prováděním změn. V agentním vývoji plán omezuje okamžité přecházení od požadavku k úpravám kódu. Rozděluje práci na kontrolovatelné kroky a předem určuje, jak bude možné ověřit, že jednotlivé části i celek splnily zadání. Plánování vytváří kontrolovatelný mezistupeň mezi požadavkem a provedením a snižuje riziko, že agent optimalizuje lokální změny bez ohledu na celek.

2.3.2 DAG (Orientovaný acyklický graf) [Directed Acyclic Graph]

Orientovaný acyklický graf (DAG) je orientovaný graf bez orientovaného cyklu, který umožňuje explicitně vyjádřit závislosti a pořadí kroků pracovního postupu. V agentním workflow může DAG modelovat kroky jako uzly a jejich povinné závislosti jako hrany. Tím lze explicitně vyjádřit například posloupnost příjem požadavku → plán → implementace → testy → schválení a zabránit spuštění navazujícího kroku před splněním jeho předpokladů. DAG převádí implicitní pořadí vícefázové úlohy na kontrolovatelnou strukturu závislostí, v níž lze navazující krok spustit až po splnění jeho předpokladů.

2.3.3 Version control (Správa verzí)

Správa verzí je řízení a sledování změn zdrojových souborů a dalších verzovaných artefaktů tak, aby bylo možné změny bezpečně větvit, slučovat, auditovat a vracet. Pro autonomní vývoj poskytuje verzovací vrstva deterministický záznam reality, který je nezávislý na interním kontextu modelu. Agent může pracovat nad izolovanou změnou, její stav porovnat s výchozí verzí a v případě neúspěchu se vrátit k známému bodu historie. Správa verzí je základní stavová a návratová vrstva agentního vývoje; model navrhuje změny, ale repozitář uchovává jejich skutečnou historii.

2.3.3.1 Git

Git je distribuovaný systém správy verzí, který uchovává historii projektu, podporuje větvení a slučování změn a umožňuje deterministický návrat k předchozím stavům repozitáře.

Při agentním vývoji poskytuje správa verzí auditovatelnou historii změn a možnost bezpečně oddělit pracovní stav od stabilní linie projektu. Chybnou nebo neúspěšnou změnu lze porovnat, vrátit nebo zahodit bez závislosti na paměti modelu. Tato práce používá distribuovaný systém [Git](#) v kombinaci s platformou [GitHub](#).

Samotný [Git](#) zde zůstává vymezen jako distribuovaný systém správy verzí. Hosting a koordinaci repozitáře popisuje [GitHub](#), zadání práce [Issue \(Úloha GitHubu\)](#) [[GitHub Issue](#)], revizní integraci [Pull Request \(Požadavek na sloučení\)](#) a automatizaci [Actions \(GitHub Actions\)](#). Git poskytuje agentnímu vývoji auditovatelnou historii a možnost bezpečně izolovat, porovnávat, slučovat nebo vracet změny bez závislosti na paměti modelu.

2.3.3.1.1 Branch (Větev repozitáře) [Repository Branch]

Větev je pojmenovaná vývojová linie v systému správy verzí, která umožňuje provádět změny odděleně od jiné linie historie a později je porovnat nebo sloučit. V navrženém procesu DarkFactory agent nepracuje přímo v hlavní větvi. Každý úkol provádí v samostatné pracovní větvi odvozené ze základní linie projektu ⁽⁵⁾.

Tento postup přináší několik praktických vlastností:

- Ochrana hlavní větve: změny se do `main` začleňují až přes definovaný integrační proces.
- Samostatná větev pro každý úkol: pracovní historie jednoho zadání je oddělena od ostatních běhů.
- Izolace mezistavů: dočasně nefunkční nebo experimentální změny zůstávají mimo hlavní větev.
- Snadné zahazení neúspěšného běhu: pracovní větev lze odstranit bez změny stabilní historie projektu.

Pokud se hlavní větev během práce posune, pracovní větev se před integrací zaktualizuje pomocí běžných mechanismů Gitu, například rebase nebo merge. Samostatná pracovní větev izoluje mezistavy a chyby agentního běhu od stabilní hlavní linie a umožňuje celý neúspěšný pokus bezpečně zahodit.

2.3.3.1.2 Merge (Sloučení větví) [Branch Merge]

Merge je operace správy verzí, která kombinuje změny nebo historii dvou vývojových linií do společného výsledného stavu; případné konflikty vyžadují explicitní vyřešení. Agent může během jednoho úkolu vytvářet více pracovních commitů při iterativním vývoji a opravách. V navrženém procesu DarkFactory se tato pracovní historie před začleněním do hlavní větve zjednoduší pomocí Squash (Sloučení commitů) [Commit Squashing] ⁽⁵⁾. Výsledkem je jeden integrační commit odpovídající jednomu dokončenému úkolu. Pomocné mezikroky zůstávají v pracovní větvi, zatímco hlavní historie zachovává výslednou změnu jako jeden celek, který lze samostatně auditovat nebo případně vrátit. Způsob sloučení určuje, jak se pracovní historie agenta promítne do stabilní větve; před integrací je vhodné oddělit užitečný výsledný stav od experimentálních mezikroků.

2.3.3.1.3 Squash (Sloučení commitů) [Commit Squashing]

Squash je operace, při níž se více po sobě jdoucích commitů nahradí jedním souhrnným commitem, obvykle za účelem zjednodušení historie před integrací změn. V agentním běhu často vzniká více pomocných commitů během iterativního ladění. Před začleněním výsledné změny lze tyto mezikroky sloučit do jednoho logického záznamu odpovídajícího dokončenému úkolu. Squash odděluje experimentální průběh práce od dlouhodobé historie projektu a umožňuje uchovat jednu logickou změnu jako jeden auditovatelný commit.

2.3.3.1.4 Issue (Úloha GitHubu) [GitHub Issue]

GitHub Issue je strukturovaný záznam požadavku, úkolu nebo chyby v repozitáři, který může nést popis, diskusi, štítky, přiřazení a vazby na změny kódu. V agentním vývojovém procesu může Issue fungovat jako explicitní vstupní specifikace úlohy. Odděluje zadání od samotné implementace a poskytuje stabilní referenční bod pro plán, pull request i následnou revizi. Issue funguje jako stabilní zdroj zadání oddělený od implementace a umožňuje pozdější plán, změny i revizi vztáhnout ke stejnému původnímu požadavku.

2.3.3.1.5 Pull Request (Požadavek na sloučení)

Pull Request je formální návrh na začlenění změn z jedné větve repozitáře do druhé a společné místo pro automatizované kontroly, lidskou revizi a diskusi nad navrženými úpravami. V navrženém procesu DarkFactory tvoří **Pull Request (Požadavek na sloučení)** kontrolní hranici mezi pracovní větví agenta a hlavní historií repozitáře ⁽⁵⁾. Agent připraví změnu, její souhrn a výsledky automatických kontrol; lidský revizor následně rozhodne o přijetí, přepracování nebo zamítnutí změny v souladu s principem **HITL (Zapojení člověka do smyčky) [Human-in-the-loop]**.

Pull request soustřeďuje na jednom místě řádkový diff, popis změny, vazbu na původní zadání, výsledky automatických kontrol a revizní diskusi. Tím poskytuje společný bod pro strojové ověření i lidskou sémantickou kontrolu před integrací. Pull Request vytváří kontrolní hranici mezi samostatně připravenou změnou a jejím přijetím do hlavní historie projektu.

2.3.3.1.6 Required Checks (Požadované kontroly)

Požadované kontroly jsou automatizované kontroly, jejichž úspěšné dokončení je povinnou podmínkou pro přijetí nebo sloučení změny. V procesu DarkFactory nejsou automatické kontroly pouze informativní; vybrané kontroly jsou nastaveny jako podmínka integrace změny. Mezi požadované kontroly mohou patřit statická analýza a linter, typová kontrola a sestavení projektu a automatizované testy. Každá kontrola musí vrátit jednoznačný výsledek, který lze použít jako strojově vyhodnotitelnou podmínku před sloučením pull requestu ⁽⁶⁾. Required checks mění CI z informativní zpětné vazby na technicky vynucovanou podmínku integrace.

2.3.3.1.7 Branch Protection (Ochrana větví)

Ochrana větví je sada pravidel repozitáře, která omezuje přímé změny chráněných větví a vynucuje schválení, kontroly nebo jiné podmínky před sloučením. GitHub poskytuje pravidla ochrany větví *Branch Protection Rules* ⁽⁵⁾, která zabráňují začlenění neověřeného kódu do stabilní větve `main`.

- Povinné schválení člověkem: Požadavek na explicitní autorizaci kódu lidským vývojářem dříve, než GitHub povolí sloučení do produkční větve.
- Ochrana větví převádí procesní pravidla integrace do technicky vynucované bariéry, kterou agent ani člověk nemůže obejít běžným přímým zápisem.

2.4 CI (Průběžná integrace) [Continuous Integration]

Průběžná integrace (CI) je vývojová praxe, při níž se změny často integrují a automaticky ověřují sestavením, testy a dalšími kontrolami.

Kód vytvořený jazykovým modelem nelze považovat za ověřený pouze proto, že byl úspěšně vygenerován. Kontinuální integrace proto poskytuje externí a opakovatelnou kontrolu sestavení, testů a dalších strojově vyhodnotitelných podmínek ⁽⁶⁾.

Konkrétní automatizační platformu popisuje **Actions (GitHub Actions)** ⁷, izolaci běhu **Container (Softwarový kontejner) [Software Container]** ⁸ a problematiku nedeterministických selhání **Flaky Test (Nestálý test)** ⁹. Koncept CI zde zůstává zaměřen na integrační kontrakt a strojově ověřitelnou zpětnou vazbu.

Riziko nestálých testů je vyčleněno do samostatného konceptu **Flaky Test (Nestálý test)** ⁹. V agentním vývoji CI poskytuje strojově ověřitelnou zpětnou vazbu, která odděluje generování změny od jejího objektivního ověření.

2.4.1 Actions (GitHub Actions)

GitHub Actions je automatizační platforma GitHubu pro spouštění deklarovaných workflow a jejich jobů v reakci na události repozitáře nebo ruční spuštění. GitHub Actions spouští deklarovaná workflow v reakci na události repozitáře. V této práci představuje konkrétní automatizační prostředí, které realizuje CI kontroly a další repozitářové procesy. V této práci GitHub Actions představuje konkrétní prováděcí prostředí CI a repozitářové automatizace, nikoli samotný princip průběžné integrace.

2.4.2 Container (Softwarový kontejner) [Software Container]

Softwarový kontejner je izolované uživatelské běhové prostředí, které balí aplikaci a její závislosti při sdílení jádra hostitelského operačního systému.

Kontejner může CI běhu poskytnout opakovatelné uživatelské prostředí s deklarovanými nástroji a závislostmi. Tím omezuje vliv lokální konfigurace vývojářského počítače, aniž by sám o sobě zaručoval úplnou bezpečnostní izolaci. Kontejner zvyšuje reprodukovatelnost běhu, ale jeho bezpečnostní izolace závisí na konkrétní implementaci a konfiguraci.

2.4.3 Flaky Test (Nestálý test)

Nestálý test je test, který může nad stejným kódem střídavě projít a selhat kvůli neterminismu, časování, prostředí nebo externím službám.

Nestálé testy oslabují roli CI jako deterministické zpětné vazby. Agent může náhodné selhání mylně interpretovat jako regresi a začít měnit správný kód; proto je vhodné zdroje nedeterminismu omezovat a podezřelé běhy ověřovat opakováním v čistém prostředí. Nestálé testy snižují informační hodnotu CI a mohou agenta vést k opravám správného kódu; podezřelé selhání proto musí být reprodukovatelné nebo explicitně označené jako nedeterministické.

3 Language Models, Chatbots, and Agents (Jazykové modely, chatboti a agenti)

Tato skupina konceptů rozlišuje samotný jazykový model od produktových rozhraní a agentních systémů, které model používají.

LLM (Velký jazykový model) [*Large Language Model*] , **Chatbot** a **Agent** označují různé systémové vrstvy. Toto rozlišení zabraňuje zaměňování schopností modelu s funkcemi, které ve skutečnosti poskytuje aplikační nebo agentní harness. Další sekce proto popisují model, jeho bezprostřední inferenční mechanismy a nadřazené systémy odděleně.

3.1 Chatbot

Chatbot je aplikační systém, který zpřístupňuje jazykový model prostřednictvím konverzační interakce.

Na rozdíl od samotného modelu může chatbot přidávat historii konverzace, multimodální vstupy, práci se soubory, webové vyhledávání nebo další nástroje. Tyto funkce patří aplikační vrstvě, nikoli modelu samotnému.

3.1.1 ChatGPT

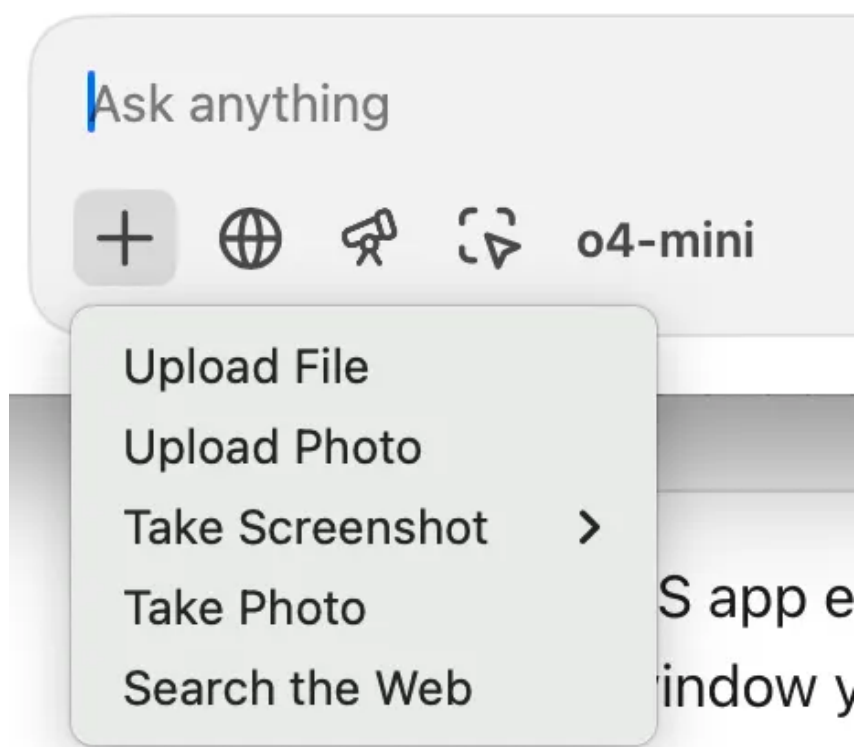
ChatGPT je konverzační produkt OpenAI zpřístupňující modely prostřednictvím chatového a aplikačního rozhraní.

Jako příklad chatbota odděluje modelovou vrstvu od uživatelského produktu: rozhraní přidává práci se soubory, obrazem, webem a dalšími nástroji, které samotný model neposkytuje.

3.1.1.1 ChatGPT App Interface (Rozhraní aplikace ChatGPT)

Oficiální snímek rozhraní aplikace ChatGPT pro macOS.

Snímek ukazuje vstupní rozhraní a nabídku práce se soubory, obrazem a webem.



Obrázek 2: Rozhraní ChatGPT pro macOS. Zdroj: OpenAI Help Center.

Příklad uživatelského povrchu nad jazykovým modelem.⁽⁷⁾ ChatGPT ilustruje rozdíl mezi jazykovým modelem a aplikačním systémem, který model obaluje.⁽⁷⁾ Chatbot je produktová vrstva nad modelem; agentní systém se od něj odlišuje především řízenou prováděcí smyčkou a schopností samostatně vykonávat akce.

3.2 Agent

Agent je softwarový systém řízený jazykovým modelem a vybavený nástroji, který samostatně vnímá stav prostředí a provádí vícezkrokové akce směřující k zadanému cíli. Rozdíl mezi *Chatbot* a agentem nespočívá nutně v použitém jazykovém modelu, ale v architektuře jeho zapojení do prostředí. Agent může číst stav repozitáře, provádět změny, spouštět nástroje a podle jejich výsledků pokračovat v další iteraci. Tuto schopnost zajišťuje nadřazená prováděcí a nástrojová vrstva, nikoli samotný model. Agent rozšiřuje model o řízené vícezkrokové jednání nad skutečným prostředím; autonomii proto určuje celý agentní systém, ne pouze model.

3.3 LLM (Velký jazykový model) [Large Language Model]

Velký jazykový model je neuronový model pro zpracování a generování posloupností tokenů; současné modely tohoto typu typicky používají *Transformer (Transformerová architektura)* [Transformer Architecture].

Při generování model opakovaně odhaduje další token na základě dosavadního kontextu. Samostatné koncepty dále popisují **Autoregressive Modeling (Autoregresivní modelování)**, tokenizaci, reprezentaci dat a omezení kontextu. Funkce jako práce se soubory, spouštění příkazů nebo správa úloh nejsou vlastnostmi jazykového modelu; poskytuje je nadřazený aplikační nebo agentní systém.

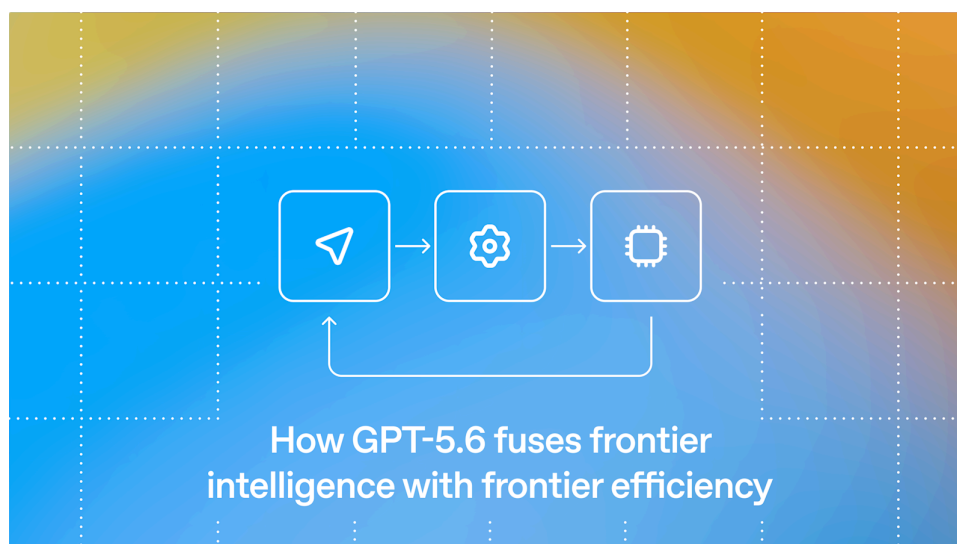
3.3.1 GPT-5.6

GPT-5.6 je modelová řada OpenAI vydaná v roce 2026 ve variantách Sol, Terra a Luna. Jde o příklad současného jazykového modelu, který je nasazován jako inferenční komponenta v produktech včetně ChatGPT a Codexu; aplikační schopnosti těchto produktů však vznikají až propojením modelu s jejich harnessy.

3.3.1.1 GPT-5.6 Product Visual (Produktový vizuál GPT-5.6)

Oficiální produktový vizuál modelové řady GPT-5.6.

Obrazový podklad je převzat z oficiálního vydání modelu OpenAI.



Obrázek 3: GPT-5.6. Zdroj: OpenAI.

Vizuální identifikace příkladu jazykového modelu.⁽⁸⁾ Příklad odděluje modelovou vrstvu od agentního systému, který model používá.⁽⁸⁾

3.3.2 Claude Opus 5

Claude Opus 5 je model společnosti Anthropic vydaný v červenci 2026.

Model je příkladem současné modelové vrstvy používané v širších agentních produktech. Samotný model není totožný s Claude Code ani desktopovou aplikací Claude; ty přidávají nástroje, stav a orchestrace.

3.3.2.1 Claude Opus 5 Product Visual (Produktový vizuál Claude Opus 5)

Oficiální produktový vizuál modelu Claude Opus 5.

Obrazový podklad je převzat z oficiálního oznámení společnosti Anthropic.



Obrázek 4: Claude Opus 5. Zdroj: Anthropic.

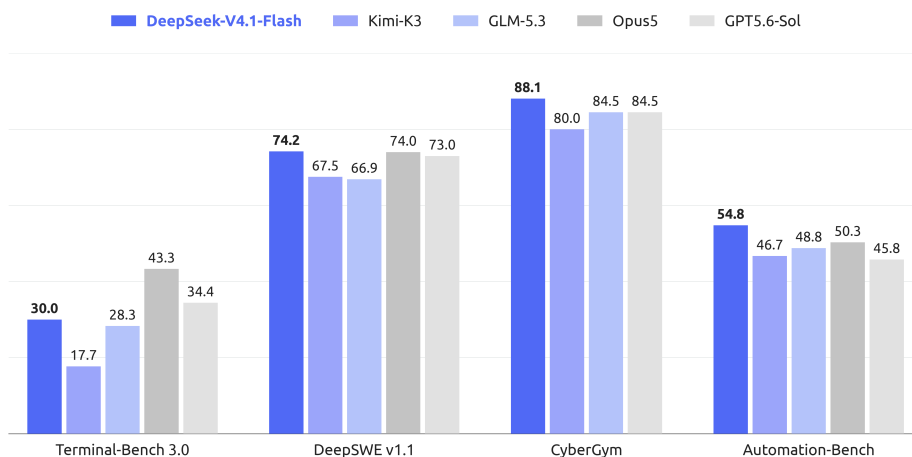
Vizuální identifikace příkladu jazykového modelu.⁽⁹⁾ Příklad zdůrazňuje rozdíl mezi modelem a agentním harness-em.⁽⁹⁾

3.3.3 DeepSeek-V4.1-Flash

DeepSeek-V4.1-Flash je multimodální model DeepSeek vydaný v září 2026. Jde o příklad odlišné současné modelové architektury: DeepSeek jej popisuje jako 552B MoE s asymetrickým Causal Encoder–Decoder uspořádáním a nativním vizuálním vstupem.

3.3.3.1 DeepSeek-V4.1-Flash Product Visual (Produktový vizuál DeepSeek-V4.1-Flash)

Oficiální produktový vizuál modelu DeepSeek-V4.1-Flash. Obrazový podklad je převzat z oficiálního vydání společnosti DeepSeek.



Obrázek 5: DeepSeek-V4.1-Flash. Zdroj: DeepSeek.

Vizuální identifikace příkladu jazykového modelu.⁽¹⁰⁾ Příklad ukazuje, že pojem jazykového modelu v agentním systému zahrnuje různé konkrétní modelové rodiny a architektury.⁽¹⁰⁾ Jazykový model je inferenční komponenta. V této práci je důležité především jeho rozhraní a omezení, protože provozní chování agenta vzniká až propojením modelu s harness-em.

3.3.4 Transformer (Transformerová architektura) [Transformer Architecture]

Transformer je architektura neuronových sítí založená na mechanismu pozornosti, která modeluje vztahy mezi prvky sekvence a tvoří základ většiny současných velkých jazykových modelů.

Současné velké jazykové modely jsou typicky realizovány transformerovou architekturou. Pro generování textu se často používá dekodérové uspořádání, které nad dosavadní sekvencí vytváří reprezentace potřebné k odhadu následujícího tokenu. Pro tuto práci je podstatné, že transformer zpracovává tokenové reprezentace v omezeném kontextu a vytváří stav potřebný k postupnému odhadu dalších tokenů; detailní matematika trénování není předmětem práce.

3.3.5 Autoregressive Modeling (Autoregresivní modelování)

Autoregresivní modelování je sekvenční postup, při němž model odhaduje následující token podmíněně na předchozí tokeny v aktuální posloupnosti. Autoregresivní generování probíhá iterativně: model zpracuje dosavadní posloupnost tokenů, vypočítá distribuci pravděpodobnosti následujícího tokenu a po jeho výběru celý krok opakuje nad rozšířenou posloupností. Autoregrese vysvětluje, proč generování probíhá po jednotlivých krocích a proč každé nové pokračování závisí na dosavadním kontextu.

3.3.6 Stochastic Decoding (Stochastické dekódování)

Stochastické dekódování je výběr výstupních tokenů z pravděpodobnostního rozdělení modelu způsobem, který může při stejném vstupu vést k různým platným pokračováním. Pravděpodobnostní dekódování znamená, že shodný vstup nemusí vždy vytvořit totožný výstup. V agentním systému proto nelze provozní spolehlivost opřít pouze o model; kritická pravidla a stavové přechody musí vynucovat deterministická vrstva harnessu. Nedeterminismus dekódování je jedním z důvodů, proč kritická provozní pravidla agentního systému musí vynucovat harness mimo model.

3.3.7 Tokenizer (Tokenizér)

Tokenizer je komponenta, která převádí text nebo jiný podporovaný vstup na posloupnost tokenů a jejich identifikátorů a provádí odpovídající zpětné dekódování. Konkrétní tokenizér určuje slovník i pravidla rozdělení vstupu, často pomocí subword metod, jako je Byte Pair Encoding. Stejný text proto může mít u různých modelových rodin odlišný počet tokenů a jinou segmentaci. Tokenizér je deterministické rozhraní mezi textovým povrchem a diskrétní sekvencí, kterou model skutečně zpracovává.

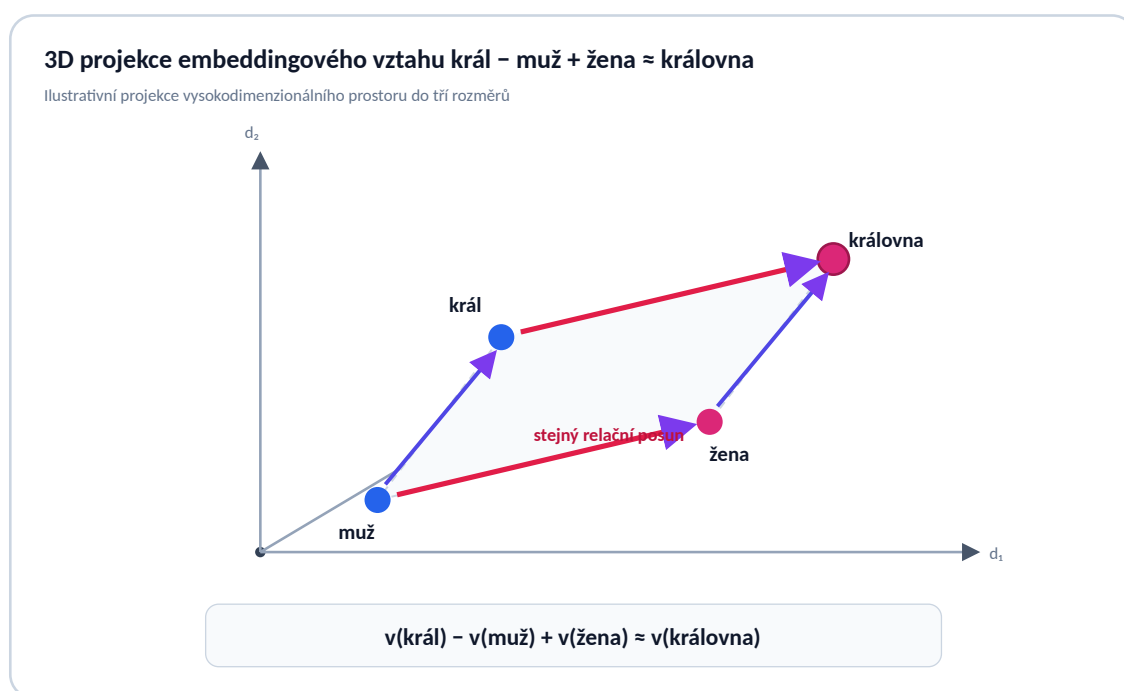
3.3.8 Token

Token je diskrétní jednotka zpracovávaná jazykovým modelem, odpovídající položce slovníku tokenizéru a reprezentovaná číselným identifikátorem.

Token nemusí odpovídat celému slovu; podle použitého tokenizéru může představovat slovo, část slova, znakový fragment nebo jinou jednotku. Po tokenizaci jsou identifikátory převedeny na vektorové reprezentace, se kterými pracují vrstvy modelu. Token je základní diskrétní jednotka modelového vstupu a výstupu a současně jednotka, podle níž se prakticky měří velikost kontextu.

3.3.9 Embedding (Vektorová reprezentace)

Embedding je vícerozměrná vektorová reprezentace tokenů nebo jiných dat, v níž numerické vztahy mezi vektory zachycují užitečné vztahy mezi reprezentovanými objekty. V jazykovém modelu embedding převádí diskrétní identifikátory tokenů na spojité vektory zpracovatelné neuronovou sítí. Podobné reprezentace lze použít také pro sémantické vyhledávání nebo porovnávání podobnosti. Známým didaktickým příkladem je přibližný relační vztah mezi vektory slov král, muž, žena a královna. Obrázek zobrazuje pouze trojrozměrnou projekci; skutečný embeddingový prostor má obvykle mnohem více rozměrů. ⁽¹¹⁾



Obrázek 6: Třírozměrná projekce příkladu vektorového vztahu král - muž + žena ≈ královna. Embedding propojuje diskrétní vstupy s numerickým prostorem, v němž lze některé vztahy mezi reprezentacemi vyjádřit směrem a vzdáleností vektorů.

3.3.10 Context Window (Kontextové okno)

Kontextové okno je maximální rozsah tokenové sekvence, kterou model při jednom inferenčním běhu dokáže zahrnout do aktivního kontextu.

Do tohoto limitu se společně započítávají systémové instrukce, uživatelský vstup, historie běhu, výsledky nástrojů a další data předaná modelu. Omezení proto přímo ovlivňuje, kolik pracovního stavu může agent udržovat současně bez výběru, externí paměti nebo kompakce. Kontextové okno je konečný pracovní prostor inference; harness musí aktivně rozhodovat, které informace v něm zůstanou.

3.3.11 KV Cache (Mezipaměť klíčů a hodnot) [Key-Value Cache]

KV Cache je mezipaměť dříve vypočtených vektorů klíčů a hodnot v pozornostních vrstvách transformeru, která při autoregresivním generování omezuje opakovaný výpočet předchozích tokenů.

Při každém novém tokenu lze znovu použít klíče a hodnoty vytvořené pro předchozí část sekvence namísto jejich úplného přepočítání. Mezipaměť tím snižuje výpočetní režii generování, ale její velikost roste s délkou aktivní sekvence a představuje významnou část paměťových nároků inference. KV Cache urychluje autoregresivní inferenci výměnou výpočetní práce za paměť, jejíž spotřeba roste s kontextem.

3.3.12 Turn (Tah interakce) [Interaction Turn]

Tah je jedna diskrétní jednotka interakce v konverzačním nebo agentním protokolu, například zpráva uživatele, výstup modelu nebo samostatně evidovaný výsledek nástroje. Agentní běh skládá sekvenci tahů, v níž se střídají vstupy prostředí, modelová rozhodnutí a výsledky provedených akcí. Historie těchto tahů tvoří část pracovního kontextu a harness rozhoduje, které z nich zůstávají modelu dostupné v dalších iteracích. Tah je protokolová jednotka agentní interakce; odděluje jednotlivé vstupy, rozhodnutí a pozorování, z nichž harness sestavuje další kontext.

4 Agentic Engineering (Agentické inženýrství)

Agentické inženýrství je disciplína zaměřená na návrh, orchestraci a provoz agentních systémů kolem jazykových modelů, včetně nástrojů, kontextu, prováděcích smyček, mantinelů a lidského dohledu.

Předmětem agentického inženýrství není samotná modelová architektura, ale systémové prostředí, které převádí modelový výstup na řízené jednání. Patří sem zejména návrh agentního harnessu, správa kontextu a stavu, rozhraní nástrojů, prováděcí smyčky, ověřování výsledků, izolace a rozhodovací body pro lidský dohled. Agentické inženýrství hodnotí celý systém kolem modelu podle toho, zda dokáže modelové schopnosti převést na opakovatelné, kontrolovatelné a ověřitelné chování.

4.1 Vibe Coding (Vibe coding)

Vibe Coding je způsob tvorby softwaru, při kterém člověk popisuje požadované chování modelu přirozeným jazykem, zkouší vzniklý výsledek a dalšími prompty jej upravuje, aniž by průběžně kontroloval samotný vygenerovaný kód.

Označení zavedl Andrej Karpathy v příspěvku na síti X v únoru 2025, kde tento způsob práce přímo pojmenoval jako „vibe coding“. Dobový záznam citátu a odkazu na původní příspěvek zachycuje také Simon Willison.^(12, 13)

Pro experimenty a malé jednorázové aplikace může být tento způsob velmi rychlý. Pro dlouhodobě udržovaný software však nestačí samotná schopnost rychle generovat změny; důležitá zůstává kontrola požadavků, testů, verzí a výsledného chování systému. Vibe Coding ukazuje nejméně kontrolovaný konec spektra práce s agentní AI a pomáhá odlišit rychlé generování od řízeného agentního softwarového inženýrství.

4.2 Slop

Slop je neformální označení pro velmi nekvalitní digitální obsah, zejména obsah vytvořený umělou inteligencí.

Cambridge Dictionary uvádí pojem AI slop pro nekvalitní digitální obsah vytvořený AI.⁽¹⁴⁾ V kontextu softwaru lze stejný problém pozorovat jako rychle vytvořený výstup, který na první pohled splňuje zadání, ale obsahuje zbytečné vrstvy, duplicity, neověřené předpoklady nebo obtížně udržovatelný kód. Agentní workflow proto musí hodnotit výsledek podle testů, architektury a skutečného stavu repozitáře, ne podle množství vygenerovaného textu nebo kódu. Pojem Slop pojmenovává riziko, že vysoká rychlost generování vytvoří mnoho výstupu bez odpovídající kvality a ověření.

4.3 Prompt Engineering (Promptové inženýrství)

Promptové inženýrství je systematický návrh a strukturování instrukcí, které vymezují požadované chování, kontext a provozní očekávání jazykového modelu nebo agenta.

V agentním systému prompt určuje roli modelu, způsob práce s dostupným kontextem a očekávaný postup při používání nástrojů. Instrukce však nejsou technicky vynucovanou bezpečnostní hranicí: podmínky, jejichž porušení by mohlo poškodit stav systému nebo obejít procesní pravidla, musí zajišťovat deterministický **Guardrail (Deterministický mantinel) [Deterministic Guardrail]**. Základní dlouhodobé instrukce jsou soustředěny v **System Prompt (Systémový prompt)**. Promptové inženýrství řídí model prostřednictvím instrukcí; kritická pravidla, která musí platit bez ohledu na modelový výstup, patří do kódu harnessu.

4.3.1 System Prompt (Systémový prompt)

Systémový prompt je instrukční vrstva s vysokou prioritou, která vymezuje roli, chování, dostupné prostředky a provozní očekávání jazykového modelu nebo agenta. V agentním harnessu tvoří systémový prompt stabilní základ instrukčního kontextu. Může definovat roli agenta, pracovní postup, způsob používání nástrojů a pravidla komunikace, ale nemůže nahrazovat oprávnění, validaci nástrojů ani jiné deterministické mechanismy. Systémový prompt je hlavní instrukční rozhraní harnessu k modelu; určuje očekávané chování, nikoli technicky vynucenou bezpečnostní hranici.

4.4 Agent Harness (Agentní harness)

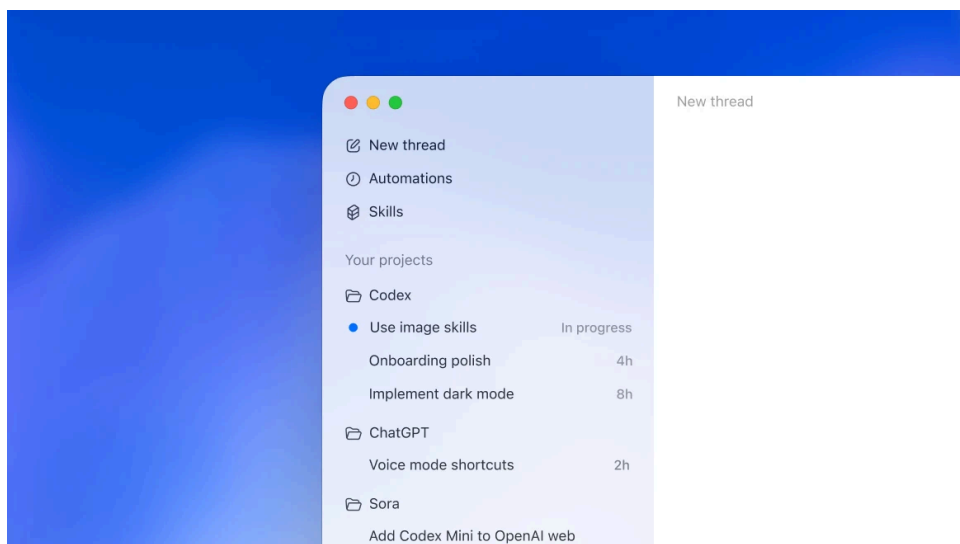
Agentní harness je aplikační a orchestrační vrstva, která propojuje model s prostředím a řídí jeho opakované jednání nad stavem a nástroji. Harness sestavuje pracovní kontext, zpřístupňuje nástroje, spravuje stav úlohy, přijímá výsledky provedených akcí a určuje, kdy může agent pokračovat nebo kdy má běh skončit. Ústředním prováděcím mechanismem je **Agent Loop (Smyčka ReAct) [ReAct Loop]**.

4.4.1 Codex

Codex je agentní vývojový systém OpenAI určený pro samostatné plnění softwarově-inženýrských úloh. Desktopová aplikace umožňuje řídit více agentů paralelně, oddělovat jejich práci do vláken a worktree, kontrolovat diffy a delegovat dlouhotrvající úlohy. Tím představuje praktický příklad harnessu, který rozšiřuje model o stav, nástroje, izolaci a pracovní postup.

4.4.1.1 Codex App Interface (Rozhraní aplikace Codex)

Oficiální produktový snímek aplikace Codex. Snímek dokumentuje specializované rozhraní pro práci s agentními úlohami a dovednostmi.



Obrázek 7: Aplikace Codex. Zdroj: OpenAI.

Vizuální příklad agentního vývojového prostředí.⁽²⁾ Codex ukazuje posun od generování fragmentů k řízenému agentnímu vývojovému procesu.⁽²⁾

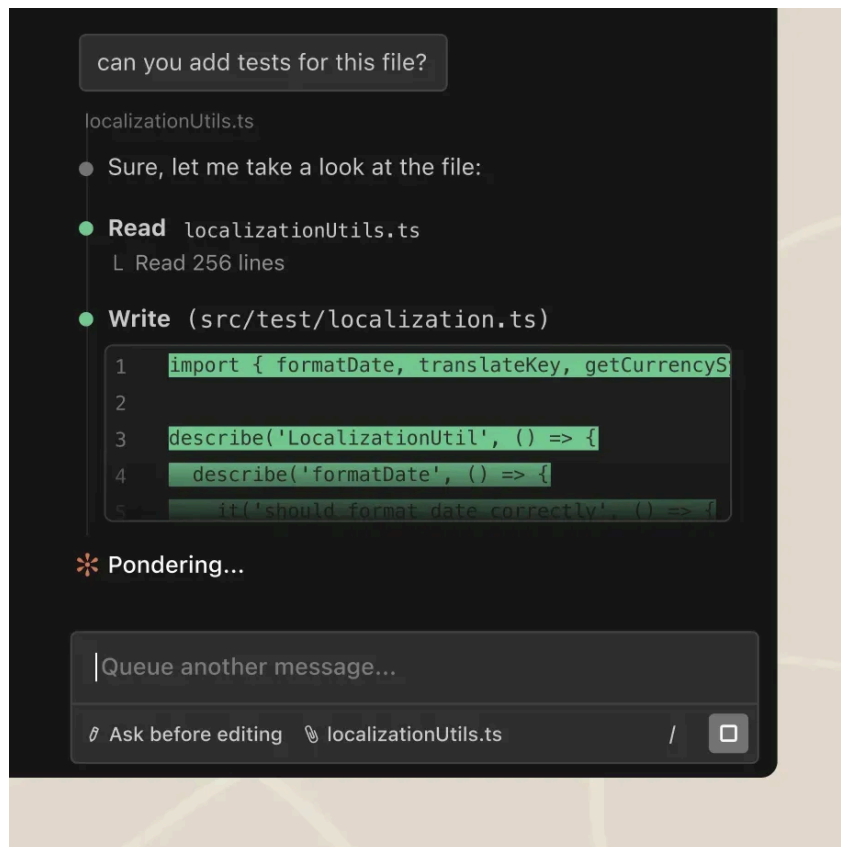
4.4.2 Claude Code

Claude Code je agentní vývojové prostředí společnosti Anthropic dostupné v terminálu, IDE, na webu a v desktopové aplikaci.

Systém může plánovat změny, číst a zapisovat soubory, spouštět příkazy a testy, používat Git a MCP a vytvářet pull requesty. Tyto schopnosti vznikají kombinací modelu s nástrojovou a stavovou vrstvou harnessu.

4.4.2.1 Claude Code Interface (Rozhraní Claude Code)

Oficiální snímek rozhraní Claude Code při práci se zdrojovým kódem. Snímek zobrazuje čtení souboru, zápis změny a průběžný stav agentní úlohy.



Obrázek 8: Claude Code při práci se souborem. Zdroj: Anthropic.

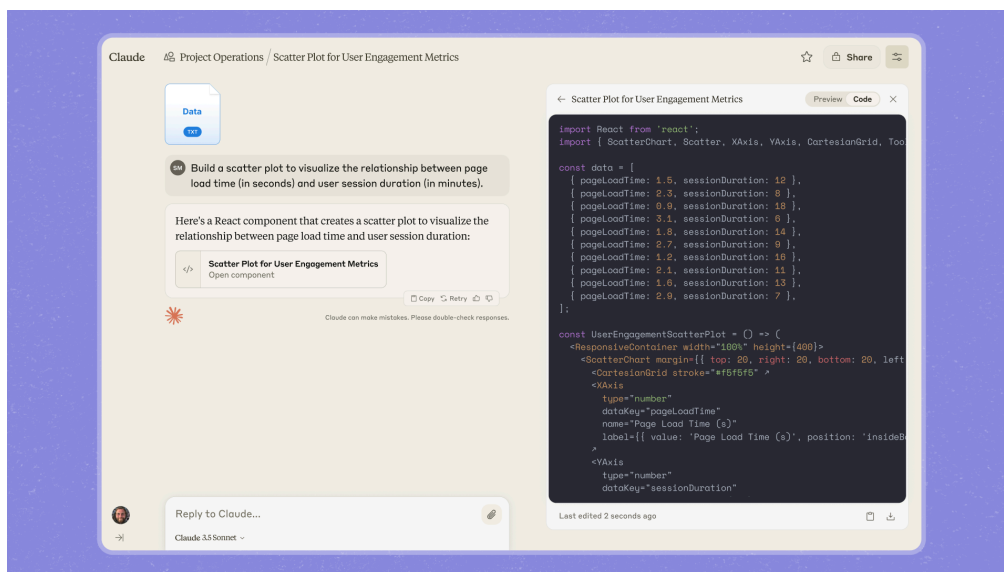
Vizuální příklad nástrojově řízeného kódovacího agenta.⁽³⁾ Claude Code je konkrétní příklad agentního harnessu propojeného s reálným vývojovým prostředím.⁽³⁾

4.4.3 Claude Desktop (Aplikace Claude pro desktop) [Claude Desktop App]

Claude Desktop je desktopová aplikační vrstva nad modely Claude, která propojuje konverzaci s lokálními soubory, aplikacemi, webem a dalšími schopnostmi. Anthropic popisuje desktopovou aplikaci jako prostředí, v němž může Claude se souhlasem uživatele přímo číst, upravovat a ukládat soubory a pracovat s dalšími aplikacemi. Jde proto o širší příklad harnessu než samotný chatbot.

4.4.3.1 Claude Desktop Interface (Rozhraní aplikace Claude)

Oficiální snímek aplikačního rozhraní Claude s pracovním artefaktem. Snímek ukazuje, že aplikační vrstva může vedle konverzace zpřístupnit soubory, nástroje a samostatný pracovní povrch.



Obrázek 9: Aplikační rozhraní Claude. Zdroj: Anthropic.

Vizuální příklad desktopového aplikačního harnessu.⁽¹⁵⁾ Desktopová aplikace ukazuje, jak harness rozšiřuje model o oprávnění, kontext a akce v uživatelském prostředí.⁽¹⁵⁾ Schopnosti agentního systému nevznikají pouze v modelu. Harness modelu poskytuje prostředí a pravidla, díky kterým lze jeho rozhodnutí převádět na řízené akce.

4.4.4 Sandbox (Izolované běhové prostředí)

Sandbox je omezené běhové prostředí určené k oddělení prováděného kódu a jeho oprávnění od hostitelského systému. Přímé spouštění kódu umožňuje agentovi vytvářet skripty a příkazy, které harness provádí v řízeném prostředí. Flexibilita tohoto přístupu současně zvětšuje bezpečnostní význam hranice mezi generovaným kódem, pracovními soubory, sítí, tajnostmi a hostitelským systémem. Běžné linuxové kontejnery využívají izolační mechanismy hostitelského jádra a samy o sobě nepředstavují stejnou bezpečnostní hranici jako samostatný virtualizovaný kernel. Dokumentace gVisoru proto výslovně rozlišuje běžnou kontejnerovou izolaci od silnějších sandboxových přístupů a upozorňuje, že samotné mechanismy sdíleného jádra je nutné správně omezit⁽¹⁶⁾. Pro nedůvěryhodný modelově generovaný kód má harness kromě procesní izolace explicitně řídit mounty, oprávnění, síť a dostupnost tajností. U rizikovějších úloh může použít silnější hranici, například microVM typu Firecracker⁽¹⁷⁾. Bezpečnost sandboxu proto není vlastností jediného runtime přepínače. Je výsledkem kombinace izolační technologie a minimálních oprávnění, omezeného souborového a síťového přístupu, řízení tajností a pozorovatelného životního cyklu spuštěného procesu. Bezpečné spouštění modelově generovaného kódu vyžaduje vynutitelnou izolaci procesů, souborů, oprávnění, tajností a sítě; podle rizika může být nutná silnější hranice než běžný kontejner.

4.4.5 Guardrail (Deterministický mantinel) [Deterministic Guardrail]

Guardrail je programově vynucené omezení nebo kontrola, která neponechává kritické provozní pravidlo pouze na pravděpodobnostním rozhodnutí modelu. Pokud samotná instrukce v promptu neposkytuje dostatečnou záruku, harness může pravidlo vynutit deterministicky, například omezením přístupových práv, validací parametrů nástroje nebo odmítnutím nepovoleného stavového přechodu. Kritická bezpečnostní a procesní pravidla patří do deterministicky vynucované vrstvy harnessu, nikoli pouze do textových instrukcí modelu.

4.4.6 HITL (Zapojení člověka do smyčky) [Human-in-the-loop]

Zapojení člověka do smyčky (HITL) je návrhový vzor, v němž lidský operátor zůstává součástí rozhodovacího procesu prostřednictvím explicitních schvalovacích bran, zejména před významnými nebo nevratnými operacemi.

Základním principem navrženého řešení není nekritická plná autonomie, nýbrž efektivní kooperace člověka a stroje (HITL (Zapojení člověka do smyčky) [Human-in-the-loop] ⁽¹⁸⁾). Autonomnímu systému náleží mechanické a rutinní úkony, zatímco klíčová architektonická a nevratná rozhodnutí zůstávají pod kontrolou vývojáře.

Řízení lidského dohledu staví na několika explicitních bodech. První brána se týká záměru a plánu před zahájením zásahů do repozitáře. Druhá brána se týká sémantické kontroly výsledného diffu před začleněním do hlavní větve. Mezi nimi může harness provádět rutinní kroky samostatně, pokud jsou jejich účinky dohledatelné a ověřitelné.

Moderní code review je kognitivně náročná činnost a porozumění změně patří mezi její hlavní obtíže. Experimentální výsledky zároveň ukazují, že účinek podrobného vedení závisí na složitosti změny a že systematická kontrola může v některých případech snížit kognitivní zátěž ⁽¹⁹⁾. Z toho plyne praktický požadavek na harness: lidské brány nemají být zahlceny každou interní iterací agenta. Změny je vhodné předkládat v sémanticky souvisejících celcích, doplnit vysvětlením netriviálních rozhodnutí a zvýraznit zásahy do kritických částí systému.

Dohledatelnost původního zadání zajišťuje trvalé uchování doslovného požadavku v GitHub Issue. Při selhání se nemá modelové vysvětlení zaměňovat za důkaz úspěchu; harness má předat pozorovatelný stav, například diff, chybové hlášení, výsledek kontrol a stav běhu. Lidský dohled je nejúčinnější v několika explicitních branách nad srozumitelnými a ověřitelnými změnami, nikoli v mikromanagementu každé agentní iterace.

4.4.7 Plugins (Rozšíření)

Plugin je programové rozšíření běžící přímo v prostředí harnessu, které může doplnit exekuční jádro o systémové adaptéry, ovladače nástrojů nebo deterministické záchytné body.

Plugin rozšiřuje samotný harness programovým modulem. Na rozdíl od instrukční dovednosti tak může přidávat systémové adaptéry, ovladače nástrojů nebo deterministické zásahy přímo do běhové vrstvy. Plugin patří do exekuční vrstvy harnessu a používá se tehdy, když rozšíření vyžaduje programové chování namísto pouhých instrukcí pro model.

4.4.8 Agent Loop (Smyčka ReAct) [ReAct Loop]

Iterativní prováděcí cyklus autonomního agenta založený na vzoru ReAct (*Reasoning + Acting*), v němž model střídá rozhodování, volání nástrojů a vyhodnocování výsledků z běhového prostředí.

Agentní smyčka (*Agent Loop*) představuje výkonné jádro celého agent harnessu. Zatímco pasivní konverzační chatbot jednorázově odpoví na uživatelský dotaz a čeká na další vstup, agentní smyčka autonomně udržuje kontinuální iterativní proces, v němž harness opakovaně vyhodnocuje stav repozitáře, volá jazykový model a vykonává požadované systémové akce.

Provozní odpovědnosti smyčky jsou odděleny do samostatných konceptů: *Agent Session (Agentní sezení)*, *Tool Calling (Vývolávání nástrojů)*, *Loop Engineering (Inženýrství prováděcí smyčky) [Execution-loop Engineering]* a *Guardrail (Deterministický mantinel) [Deterministic Guardrail]*. Samotná agentní smyčka zde popisuje jejich iterativní koordinaci.

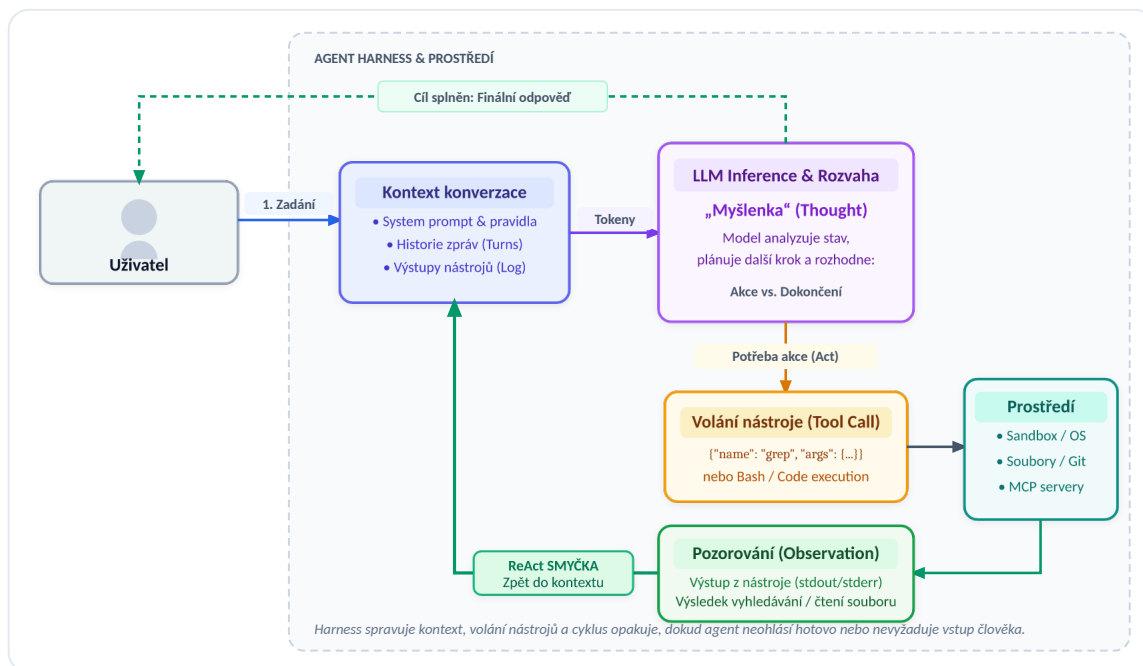
Vnitřní kognitivní krok modelu uvnitř smyčky se řídí operačním vzorem ReAct (*Reasoning + Acting*)⁽²⁰⁾, který propojuje rozvahu s přímým jednáním. Tento prováděcí cyklus sestává ze čtyř navazujících fází znázorněných na Obrázek 10:

1. Rozvaha (*Thought*): Model vyhodnotí aktuální stav kontextu a formuluje svůj nejbližší záměr.
2. Volání nástroje (*Tool Call*): Emitování strukturovaného požadavku na provedení konkrétní akce s určenými parametry.
3. Vykonání a pozorování (*Observation*): Harness bezpečně provede akci v systému a výstup (výpis souboru či chybovou zprávu) vloží zpět do kontextu.
4. Navazující iterace: Model v dalším tahu analyzuje získanou odezvu a rozhoduje o dalším kroku.

Kvalita a provozní spolehlivost celého systému tak závisí v první řadě na robustnosti architektury harnessu a spolehlivosti jeho agentní smyčky, nikoliv pouze na samotném jazykovém modelu.

4.4.8.1 ReAct Agent-loop Diagram (Schéma agentní smyčky ReAct)

Schéma znázorňuje iterativní tok mezi uživatelem, aktivním kontextem, jazykovým modelem, voláním nástrojů a pozorováním výsledků v agentní smyčce ReAct. Diagram odděluje modelové rozhodnutí od exekuce nástroje a zpětného vložení pozorování do další iterace, což odpovídá základnímu vzoru Reasoning + Acting.



Obrázek 10: Architektura autonomní ReAct smyčky a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.

Vizuální tok ukazuje, že agentní běh je uzavřená iterace modelového rozhodnutí, externí akce a nového pozorování. ⁽²⁰⁾ Agentní smyčka koordinuje opakované rozhodování modelu s vykonáváním akcí; její provozní spolehlivost proto závisí na řízení stavu, nástrojů, rozpočtů a podmínek ukončení.

4.4.8.2 Agent Session (Agentní sezení)

Agentní sezení je ohraničený běh agenta se sdíleným stavem, systémovými instrukcemi, pracovním kontextem, historií tahů a provozními rozpočty. Správa agentního sezení zahrnuje sestavení systémového promptu, načtení relevantního kontextu repozitáře, průběžné uchování stavu a sledování rozpočtů, například spotřeby tokenů nebo počtu iterací. Sezení poskytuje jednotku životního cyklu, ve které lze konzistentně spravovat stav, kontext a zdrojové limity jednoho agentního běhu.

4.4.8.3 Loop Engineering (Inženýrství prováděcí smyčky) [Execution-loop Engineering]

Inženýrství prováděcí smyčky je návrh a řízení stavových přechodů, podmínek ukončení, rozpočtů, opakování, eskalací a vazby mezi rozhodováním modelu a nástroji. Inženýrství prováděcí smyčky odděluje provozní řízení od samotného modelového rozhodování. Patří sem stavové přechody, podmínky ukončení, rozpočty běhu, opakování po selhání, detekce uvíznutí a pravidla pro eskalaci nebo lidské schválení. Loop engineering převádí otevřenou agentní iteraci na provozně ohraničený proces s explicitními pravidly pokračování, selhání a ukončení.

4.4.8.4 Agent Divergence Pathologies (Patologie divergence)

Divergence je třída selhání agentní smyčky, při níž se iterativní běh vzdaluje cíli například perseverací, oscilací nebo nekontrolovanou spotřebou zdrojů.

Ponechání jazykového modelu v neomezené prováděcí smyčce vede k předvídatelným selháním⁽²¹⁾. V důsledku autoregresivní povahy se v kontextu snadno vytvoří pravděpodobnostní atraktor, který model uvězní v neproduktivním cyklu.

Mezi typické patologie patří:

- Perseverace a zacyklení: Opakované volání identického nástroje se stejnými neplatnými argumenty (např. čtení neexistujícího souboru) i po obdržení chybové zprávy.
- Oscilace a těkání (*Thrashing*): Střídavé přepínání mezi dvěma protichůdnými zásahy (úprava modulu A rozbije modul B a následná oprava B rozbije modul A).
- Nekontrolovaná spotřeba zdrojů (*Context Runaway*): Rychlé vyčerpání kontextového okna i finančního rozpočtu na volání API bez dosažení cíle.

Divergenci nelze řešit pouze lepším promptem; harness musí rozpoznávat neproduktivní trajektorie a omezovat jejich pokračování.

4.4.9 Tool Calling (Vyvolávání nástrojů)

Vyvolávání nástrojů je mechanismus, kterým model požádá okolní systém o provedení konkrétní externí akce a předá jí potřebné parametry.

Nástroj zpřístupňuje modelu operaci, kterou samotná textová inference neprovádí, například čtení souboru, dotaz na API nebo změnu stavu systému. Harness přijme požadavek modelu, zkontroluje jej, provede příslušnou operaci a vrátí výsledek zpět do dalšího kroku agentního běhu.

4.4.9.1 JSON Schema Tool Calling (Vyvolávání nástrojů s JSON Schema)

JSON Schema Tool Calling popisuje parametry nástroje formálním JSON Schema a před provedením ověřuje, že vygenerované argumenty odpovídají očekávané struktuře.

JSON Schema je standardní formát pro popis struktury a validaci JSON dat. Při vyvolávání nástrojů tak může harness přesně určit povinná pole, datové typy a další omezení vstupu. Moderní rozhraní modelů mohou navíc generování argumentů přímo omezit tak, aby odpovídalo dodanému schématu.^(22, 23) Schéma dává volání nástroje strojově kontrolovatelný kontrakt mezi modelem a kódem, který akci skutečně provádí.

4.4.9.2 Code Execution (Spouštění kódu)

Spouštění kódu je nástrojová schopnost, která agentovi umožňuje vykonat program nebo příkaz a získat jeho skutečný výstup.

V současných agentních systémech se tato schopnost typicky poskytuje přes řízené shellové nebo programové prostředí. Například dokumentace Anthropic popisuje provádění Bash a Python operací v sandboxovaném kontejneru včetně práce se soubory.⁽²⁴⁾

Pro vývoj softwaru umožňuje Code Execution spouštět testy, formátovače, buildy, diagnostické příkazy a další ověřovací kroky místo toho, aby model jejich výsledek pouze odhadoval. Code Execution umožňuje agentovi ověřovat změny skutečným během programu a vracet pozorovaný výsledek zpět do agentní smyčky. Tool Calling vytváří rozhraní mezi rozhodnutím modelu a skutečnou operací provedenou mimo model.

4.4.9.3 MCP (Model Context Protocol)

Model Context Protocol (MCP) je otevřený standard původně navržený společností Anthropic pro standardizovanou komunikaci AI aplikací s externími nástroji, zdroji a daty prostřednictvím zpráv JSON-RPC.

Pro sjednocení rozhraní mezi AI aplikacemi a externími nástroji či datovými zdroji vznikl otevřený standard **MCP (Model Context Protocol)** ⁽²⁵⁾. Namísto vytváření proprietárních rozhraní pro každou službu definuje MCP standardizovaný způsob komunikace.

Základní vlastnosti protokolu MCP:

- Protokolové rozhraní: Komunikace probíhá prostřednictvím standardu JSON-RPC (přes standardní vstup/výstup `stdio` nebo proud událostí `Server-Sent Events / SSE`).
- Architektonické oddělení: Implementace nástrojů běží jako samostatný proces mimo jádro harnessu. MCP servery fungují jako znovupoužitelné komponenty, které lze snadno sdílet napříč různými agenty a projekty.

MCP odděluje implementaci integrací od jádra harnessu a umožňuje stejné nástroje a datové zdroje znovu používat napříč agentními systémy.

4.4.10 Skills (Dovednosti)

Skill je znovupoužitelný balíček instrukcí a volitelných zdrojů, který agent načte tehdy, když je relevantní pro řešení úkol.

Otevřený formát Agent Skills používá adresář se souborem `SKILL.md`. Tento soubor obsahuje metadata a vlastní instrukce; adresář může navíc obsahovat skripty, reference nebo další zdroje. ⁽²⁶⁾

Smyslem je přesunout specializované postupy mimo základní systémový prompt. Agent tak může mít k dispozici větší množství schopností, aniž by musel jejich úplné instrukce držet v kontextu po celou dobu práce.

4.4.10.1 SKILL.md (Formát SKILL.md) [SKILL.md Format]

`SKILL.md` je povinný definiční soubor Agent Skill. Podle specifikace obsahuje YAML frontmatter následovaný instrukcemi v Markdownu; povinnými poli frontmatteru jsou `name` a `description`. ⁽²⁶⁾

Následující úplný příklad používá povinná pole i několik volitelných polí povolených specifikací: ⁽²⁶⁾

```
---
name: repository-tests
```

```

description: Run, diagnose, and summarize the repository test suite. Use
when a software change must be validated before completion.
license: MIT
compatibility: Requires the repository's test runner to be available in
the runtime.
metadata:
  author: DarkFactory
  version: "1.0"
allowed-tools: Bash Read
---

# Repository tests

Use this skill when a change must be validated against the repository's
tests.

## Procedure

1. Detect the test command documented by the repository.
2. Run the smallest relevant test scope first.
3. Fix or report failures before continuing.
4. Run the complete required suite before completion.
5. Report the commands run and the final result.

## Failure reporting

For every unresolved failure, include the command, failing target, and
relevant output.

```

Specifikace dovoluje vedle `SKILL.md` také volitelné adresáře například pro skripty, reference a assety; ty se načítají pouze podle potřeby. ⁽²⁶⁾ Formát `SKILL.md` odděluje stručná metadata potřebná pro nalezení dovednosti od detailních instrukcí, které agent načte až při jejím použití. Skills modularizují opakovatelné postupy a umožňují načítat specializované instrukce až ve chvíli, kdy jsou potřeba.

4.4.10.2 Progressive Disclosure (Postupné zpřístupňování kontextu)

Postupné zpřístupňování je strategie, při níž se do aktivního kontextu nejprve vkládají pouze stručné popisy schopností a podrobné instrukce se načtou až při jejich použití.

U dovedností snižuje postupné zpřístupňování kontextovou režii: základní prompt obsahuje pouze přehled dostupných dovedností a úplný obsah příslušného `SKILL.md` se načte až tehdy, když jej agent pro konkrétní úlohu potřebuje. Do aktivního kontextu se dostávají pouze právě potřebné instrukce, čímž se omezuje režie bez ztráty dostupnosti specializovaných postupů.

4.4.10.3 Script (Skript)

Skript je soubor nebo posloupnost příkazů určených k automatizovanému vykonání interpretem, shellem nebo jiným běhovým prostředím. Skript poskytuje deterministickou exekuci pro úlohy, u nichž není vhodné znovu rozhodovat pomocí jazykového modelu, například pro opakovatelné transformace, validace nebo obslužné kroky dovednosti. Skripty přesouvají opakovatelné deterministické kroky mimo pravděpodobnostní rozhodování modelu.

4.4.10.4 Hook (Událostní záchytný bod) [Event Hook]

Hook je definovaný bod životního cyklu nebo události, na který lze navázat vlastní deterministickou logiku před, po nebo místo standardního chování systému. Hook váže deterministickou logiku na konkrétní událost životního cyklu harnessu, například před spuštěním nástroje, po dokončení kroku nebo při změně stavu běhu. Hook umožňuje vynutit opakovatelné chování v přesně určeném okamžiku bez toho, aby model musel stejné pravidlo znovu odvozovat.

4.4.11 Context Engineering (Kontextové inženýrství)

Kontextové inženýrství je systematický návrh, výběr, pořadí a životní cyklus informací zpřístupňovaných modelu v aktivním kontextu. Kontext zahrnuje systémové instrukce, pracovní historii, výsledky nástrojů, externě načtená data a další informace, které model používá při rozhodování. Návrh této vrstvy určuje, které informace jsou modelu dostupné, kdy se načítají, jak dlouho zůstávají aktivní a jak se nahrazují při omezené kapacitě kontextového okna. Kontextové inženýrství řídí informační prostředí modelu; cílem není maximalizovat objem kontextu, ale udržet relevantní, přesný a provozně použitelný stav.

4.4.11.1 Compaction (Kompakce kontextu) [Context Compaction]

Kompakce kontextu je zmenšení aktivního kontextu výběrem, shrnutím nebo nahrazením starší historie kompaktnější reprezentací tak, aby se běh vešel do kontextového okna. Správa aktivního kontextu je součástí kontextového inženýrství (Context Engineering (Kontextové inženýrství)). Při rozsáhlejších úlohách se kontextové okno nevyhnutelně zaplní. V okamžiku, kdy objem historie dosáhne kritické hranice, musí agent harness přistoupit ke kompakci kontextu *compaction* ⁽²⁷⁾ — model je vyzván, aby dosavadní průběh sezení zkrátil do syntetického souhrnu, který nahradí starší část historie.

Tento proces však představuje destruktivní ztrátovou kompresi:

- Ztráta deterministických detailů: Model při rekurzivním zkracování vynechává přesná čísla řádků, signatury privátních funkcí, přesné cesty k souborům a doslovná chybová hlášení kompilátoru.
- Oslabení negativních pravidel: Explicitní zákazy (např. neměnit veřejné rozhraní API) bývají v souhrnu zevšeobecněny nebo zcela vypuštěny.
- Konfirmační zkreslení (*Confirmation Bias*): Model v souhrnu upřednostňuje fakta odpovídající jeho vnitřním statistickým asociacím na úkor netriviálních specifik konkrétního projektu.

Kompakce prodlužuje běh za cenu ztráty informace, a proto musí být navržena tak, aby neodstraňovala detaily nutné pro další rozhodování.

4.4.11.2 RAG (Generování rozšířené vyhledávání) [Retrieval-Augmented Generation]

Retrieval-Augmented Generation (RAG) je architektura, v níž systém před generováním nebo během něj vyhledá relevantní informace z externího zdroje a vloží je do kontextu modelu.

Aby se předešlo ztrátě informací způsobené kompakcí, moderní agentní architektury přesouvají část paměti mimo samotné kontextové okno. Namísto spoléhání se na jediný lineární textový kontext se uplatňují strukturovaná externí úložiště.

K hlavním přístupům patří:

- Hierarchická epizodická paměť (RAG⁽²⁸⁾): Ukládání doslovných protokolů nástrojů a historie úloh do externí databáze; do kontextu se selektivně injektují pouze bezprostředně relevantní fragmenty.
- Persistentní graf stavu projektu (*Project State Graph*): Udržování explicitního, strukturovaného přehledu o stavu repozitáře (seznam modifikovaných souborů, otevřené úkoly, výsledky testů a platné invarianty) mimo kontextové okno.

Díky tomu může agent kdykoliv obnovit přesný stav projektu bez závislosti na ztrátovém rekurzivním shrnování. RAG odděluje dlouhodobé uchování informací od omezeného aktivního kontextu a umožňuje načítat pouze data relevantní pro aktuální krok.

4.4.11.3 Context Rot (Degradace kontextu)

Degradace kontextu je pokles schopnosti modelu spolehlivě využívat informace v dlouhém, hlučném nebo vzájemně si konkurujícím kontextu.

Schopnost jazykového modelu pracovat s dlouhým kontextem nelze posuzovat pouze podle nominální velikosti okna. Ačkoliv moderní modely deklarují kapacitu statisíců tokenů, jejich schopnost efektivně vyhledávat a logicky propojovat fakta s rostoucí délkou kontextu výrazně klesá. Tento jev se označuje jako **Context Rot (Degradace kontextu)**.

V praxi se projevuje dvěma hlavními mechanismy:

- Lost in the Middle ⁽²⁹⁾: Pozornostní vrstvy transformeru spolehlivě vnímají informace na samém začátku a konci okna, zatímco fakta umístěná uprostřed dlouhého textu jsou často přehlížena.
- Multi-Needle Reasoning: Schopnost logicky provázat několik na sobě závislých informací rozptýlených napříč různými soubory; s rostoucí délkou kontextu tato schopnost prudce klesá.

Při komplexním křížovém refaktoringu ve velkém kontextu proto model často přehlédne klíčové souvislosti, které by v menším a čistším okně zpracoval bez potíží. Nominálně větší kontextové okno samo o sobě nezaručuje lepší výkon; kvalita závisí na tom, zda model dokáže relevantní informace v kontextu skutečně najít a propojit.

4.4.11.4 Semantic Drift (Sémantický posun)

Sémantický posun je postupné zkreslování významu a faktického stavu při opakovaném ztrátovém shrnování nebo transformaci kontextu.

Pokud je historie sezení v dlouhém vývojovém běhu shrnována vícekrát po sobě, vzniká řetězec ztrátových transformací ($S_{k+1} = f(S_k, \Delta_k)$).

Rizika sémantického posunu spočívají v těchto jevech:

- Efekt tiché pošty: Drobné zkreslení či halucinace vzniklá v kole k je v kole $k + 1$ přijata jako nezpochybnitelný historický fakt.
- Divergence modelu od reality: Po několika cyklech komprese se vnitřní model reality agenta zcela rozejde se skutečným stavem zdrojového kódu v souborovém systému. Výsledkem je stav, kdy agent sebevědomě reportuje vyřešení úkolu, ačkoliv reálný kód zůstává v nefunkčním či neúplném stavu. Opakovaná komprese může změnit pracovní reprezentaci reality natolik, že další rozhodování vychází z chybných historických předpokladů.

4.4.12 Graph Engineering (Inženýrství pracovních grafů) [Workflow-graph Engineering]

Inženýrství pracovních grafů je návrh agentních nebo automatizačních workflow jako explicitních grafů uzlů, závislostí a přechodů namísto jediné neomezené smyčky. Monolitická agentní smyčka selhává při řešení komplexních, vícefázových úloh. Pro spolehlivé škálování se v moderních systémech uplatňuje hierarchická dělba práce a formalizace procesu do podoby grafu.

Škálování je dále rozloženo na dva samostatné koncepty: hierarchickou delegaci prostřednictvím **Subagent (Podřízený agent)** a explicitní závislosti pracovního postupu prostřednictvím **DAG (Orientovaný acyklický graf) [Directed Acyclic Graph]**. Grafová struktura umožňuje rozdělit složitou úlohu na kontrolovatelné kroky, oddělit jejich odpovědnosti a explicitně řídit závislosti mezi nimi.

4.4.12.1 Subagent (Podřízený agent)

Subagent je dočasná nebo specializovaná agentní instance, které nadřazený orchestrátor deleguje vymezenou dílčí úlohu a následně převezme její výsledek.

Při hierarchické dělbě práce hlavní orchestrátor rozděluje rozsáhlou úlohu a jednotlivé části deleguje specializovaným subagentům, například pro průzkum repozitáře, plánování nebo implementaci. Po dokončení dílčího běhu může nadřazený agent převzít pouze jeho výsledek namísto celé pracovní historie subagenta. Subagenti umožňují paralelní nebo specializovanou práci a současně omezují množství dílčí pracovní historie, které musí zůstat v kontextu hlavního orchestrátoru.

5 Results and Discussion (Výsledky a diskuse)

Výsledky a diskuse hodnotí, které části navržené architektury byly ve zvoleném snapshotu DarkFactory skutečně implementovány a ověřeny a které zůstávají pouze součástí dokončovaného návrhu.

Evaluační snapshot je rozdělen do samostatných konceptů: reprodukovatelný snapshot, ověřený stav implementace, odpovědi na výzkumné otázky a omezení evaluační. Tím se oddělují pozorované výsledky od tvrzení, která by vyžadovala dosud neprovedenou koncovou akceptaci. DarkFactory poskytuje konkrétní implementační důkazy pro podstatnou část navrženého harnessu, ale aktuální evidence neopravňuje tvrdit dokončenou produkční autonomii ani uzavřenou flotilovou akceptaci.

5.1 Evaluation Snapshot (Evaluační snapshot)

Evaluační snapshot je pevně určený stav implementačního repozitáře, vůči němuž jsou formulována tvrzení v této kapitole.

Hodnocení vychází z repozitáře DarkFactory ⁽³⁰⁾ na větvi `darkfactory` v commitu `d4b2fe4159ccbe9899c0cf44eadee0063441f12e` ze dne 20. září 2026. Pro tento stav byly na GitHub Actions úspěšně dokončeny workflow CI, Release a Deploy Documentation. Současně byly zaznamenány úspěšné běhy workflow Autonomous Agent a PR Approval and Auto-Merge, což dokládá funkčnost části samoobslužného vývojového toku. Snapshot není prezentován jako finální vydání produktu. Jeho účelem je přesně vymezit, ke kterému stavu zdrojového kódu a GitHub řízení se vztahuje následující hodnocení. Výsledky jsou reprodukovatelně svázány s konkrétním commitem a jeho pozorovatelným CI/GitHub stavem.

5.2 Verified Implementation State (Ověřený stav implementace)

Ověřený stav implementace zahrnuje pouze vlastnosti doložené zdrojovým kódem, úspěšnými automatickými kontrolami nebo uzavřenými implementačními požadavky v evaluačním snapshotu.

Ve snapshotu je dokončen mechanismus detekce jazyků, balíčků a domén a z něj odvozené deterministické quality akce; odpovídající Request 341 je uzavřen. Aktuální CI tuto architekturu používá příkazem `df ci matrix` a spouští detekované kontroly jako matici podle skutečného obsahu repozitáře.

Dokončena je také hranice `@darkfactory/auth` pro uživatelskou autentizaci přes GitHub App (Request 423) a nativní dokumentační cesta `docs.df` – `@darkfactory/docs` – `@darkfactory/web` (Request 424). V CLI je integrován společný registr příkazů používaný operátorskými povrchy, ale celý Request 403 pro dokončení podporovaného CLI zůstává otevřený.

Tyto výsledky ukazují, že navržené dělení odpovědností do detekce, capability vrstvy, deterministických akcí, autentizace, dokumentačního grafu a společných operátorských rozhraní není pouze teoretické. Jednotlivé části existují v produkčním stromu a procházejí automatizovanými kontrolami. Implementace prokazuje funkčnost několika klíčových stavebních bloků harnessu, nikoli však ještě úplný koncový životní cyklus požadavku.

5.3 Research Question Evaluation (Vyhodnocení výzkumných otázek)

Vyhodnocení výzkumných otázek porovnává navržené mechanismy s tím, co bylo ve snapshotu skutečně implementováno a ověřeno.

VO1 — autonomie při zachování lidského dohledu. Praktická zkušenost s DarkFactory podporuje architekturu, v níž model neprovádí změny přímo jako nedoložená textová tvrzení, ale pracuje uvnitř řízeného toku s GitHub požadavkem, plánem, deterministickými kontrolami, revizí a slučováním. Úspěšné běhy agentního a schvalovacího workflow spolu s dokončenými deterministickými quality akcemi ukazují, že významnou část rutinního toku lze automatizovat. Plný životní cyklus ve finálním TypeScript `df` engineu však ve snapshotu ještě není uzavřen, takže nelze kvantifikovat konečnou míru autonomie.

VO2 — rozpoznání neproduktivního běhu. Navržená odpověď kombinuje explicitní stav grafu, rozpočty a limity běhu, deterministické kontrolní uzly, pozorování skutečných účinků a eskalaci k člověku místo nekonečného pokračování modelové smyčky. Implementační evidence této vrstvy je pouze částečná: Request 329 pro natural-stop/result capture a Request 358 pro graph-native orchestration zůstávají otevřené. Práce proto dokládá návrh mechanismu a jeho dílčí infrastrukturu, nikoli empiricky změřenou úspěšnost detekce zacyklení v hotovém produkčním engineu.

VO3 — zachování kontextu. Architektura uchovává doslovné zadání mimo modelový kontext v GitHub Issue, odděluje trvalý stav běhu od dočasného kontextového okna a používá strukturovaný stav, kompakci a cílené znovunačítání informací. Tím omezuje závislost dlouhé úlohy na jediném rostoucím promptu. Finální důkaz dlouhodobého zachování kontextu napříč přerušením a obnovením běhu je součástí dosud neuzavřené graph-native orchestrace a flotilové akceptace. Výzkumné otázky jsou zodpovězeny na úrovni architektonických mechanismů a částečně podpořeny implementací; nejsilnější tvrzení o plné autonomii, detekci zacyklení a dlouhodobém resume zůstávají omezená stavem nedokončené koncové akceptace.

5.4 Evaluation Limitations (Omezení evaluace)

Omezení evaluace určují, která tvrzení nelze ze zvoleného snapshotu spolehlivě odvodit.

Ve snapshotu zůstávají otevřené klíčové Requesty 317 (pravdivé aktualizace větví), 329 (natural-stop/result capture), 358 (graph-native orchestration), 359 (dokončení produkčního `df` enginu), 360 (finální podporované vydání) a 361 (akceptace napříč šesti cílovými repozitáři). Otevřený zůstává i původní zastřešující Request 68.

Z toho vyplývají dvě omezení. Zaprvé, úspěšné CI a dílčí agentní běhy nejsou důkazem, že celý koncový Request → plánování → implementace → verifikace → review/fix → merge/reconciliation životní cyklus funguje ve finálním enginu bez pomocných cest. Zadruhé, tato práce neobsahuje statistický benchmark úspěšnosti agentů, ceny, latence, četnosti zacyklení ani degradace kontextu na reprezentativním souboru úloh. Taková měření by vyžadovala stabilní finální vydání a kontrolovaný experiment.

Výsledky proto mají charakter architektonické a implementační validace. Nejsou tvrzením o dokončené produkční připravenosti ani o obecně platné výkonnostní převaze DarkFactory nad jinými agentními systémy. Nejsilnější neověřená tvrzení byla z výsledků záměrně vyloučena; k jejich doložení je nutné dokončit produkční engine, finální vydání a flotilovou akceptaci.

6 Conclusion (Závěr)

Práce zkoumala, jak lze současnou agentní AI účinně zapojit do vývoje softwaru a jakou architekturu musí mít agent harness, aby spojoval vysokou míru autonomie s pozorovatelnými účinky a lidským dohledem.

Hlavního cíle bylo dosaženo na úrovni návrhu architektury a jeho významné části byly ověřeny implementací DarkFactory. Práce ukazuje, že praktické použití agentní AI není pouze otázkou schopností jazykového modelu. Klíčová je vrstva kolem modelu: správa trvalého zadání a stavu, nástrojové rozhraní, izolované provádění, detekce skutečného stavu repozitáře, deterministické kontroly, Git/GitHub workflow, řízené lidské brány a explicitní životní cyklus úlohy.

K VO1 práce ukazuje, že rutinní kroky lze přesouvat na agentní systém tehdy, když harness převádí modelová rozhodnutí na dohledatelné akce a odděluje modelové tvrzení od skutečného účinku. K VO2 navrhuje kombinaci stavového grafu, rozpočtů, kontrolních uzlů a deterministické eskalace místo neomezené monolitické smyčky. K VO3 odděluje trvalý stav a doslovné zadání od omezeného pracovního kontextu modelu a kombinuje je s kompakcí a cíleným načítáním informací.

Implementace DarkFactory potvrzuje použitelnost několika těchto principů: detekovaný quality systém, capability-resolved deterministické akce, GitHub autentizační hranice, nativní dokumentační graf a společné operátorské kontrakty existují v aktuálním kódu a procházejí automatizovanými kontrolami. Současně však evaluace ukazuje hranici současného výsledku. Ve zvoleném snapshotu ještě nejsou uzavřeny finalizace produkčního `df` enginu, finální distribuční release ani šestirepozitářová flotilová akceptace. Práce proto netvrdí, že DarkFactory již představuje plně dokončený produkční systém nebo že byla empiricky změřena konečná míra autonomie.

Výsledkem je konkrétní, implementací podpořená architektura agentního harnessu a přesně vymezený soubor mechanismů, které umožňují používat současnou agentní AI efektivněji a kontrolovatelněji při vývoji softwaru. Dalším krokem je dokončení otevřených integračních bodů, vydání finálního artefaktu a opakovatelná akceptace napříč cílovými repozitáři; teprve tato evidence umožní rozšířit závěry o kvantitativní provozní výsledky. DarkFactory podporuje závěr, že vysoká autonomie ve vývoji vyžaduje především dobře navržený harness kolem modelu; současná implementace tento návrh částečně ověřuje, ale nejsilnější tvrzení o plně dokončené produkční autonomii zůstávají správně omezená dosud neprovedenou finální akceptací.

Seznam obrázků a tabulek

Obrázek 1	AI diffusion v globálním Severu, globálním Jihu a celosvětově, H1–H2 2025. Zdroj: Microsoft AI Economy Institute ⁽⁴⁾	9
Obrázek 2	Rozhraní ChatGPT pro macOS. Zdroj: OpenAI Help Center.	19
Obrázek 3	GPT-5.6. Zdroj: OpenAI.	20
Obrázek 4	Claude Opus 5. Zdroj: Anthropic.	21
Obrázek 5	DeepSeek-V4.1-Flash. Zdroj: DeepSeek.	21
Obrázek 6	Třírozměrná projekce příkladu vektorového vztahu král – muž + žena $\approx$ královna.	23
Obrázek 7	Aplikace Codex. Zdroj: OpenAI.	27
Obrázek 8	Claude Code při práci se souborem. Zdroj: Anthropic.	28
Obrázek 9	Aplikační rozhraní Claude. Zdroj: Anthropic.	29
Obrázek 10	Architektura autonomní ReAct smyčky a tok dat mezi uživatelem, kontextem, modelem a výkonným prostředím.	32

Seznam příloh

Seznam zdrojů

1. MICROSOFT AI ECONOMY INSTITUTE. Global AI Diffusion: Q1 2026 Trends and Insights. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.microsoft.com/en-us/research/wp-content/uploads/2026/05/Microsoft-AI-Diffusion-Report-2026-Q1.pdf>
2. OPENAI. Introducing the Codex app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-the-codex-app/>
3. ANTHROPIC. Claude Code. Online. 2026. [Accessed 20 září 2026]. Available from: <https://claude.com/product/claude-code>
4. MICROSOFT AI ECONOMY INSTITUTE. Global AI Adoption in 2025—A Widening Digital Divide. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.microsoft.com/en-us/corporate-responsibility/topics/ai-economy-institute/reports/global-ai-adoption-2025/>
5. CHACON, Scott a STRAUB, Ben. *Pro Git*. 2. New York : Apress, 2014. ISBN 978-1-4842-0076-6.
6. HUMBLE, Jez a FARLEY, David. *Continuous Delivery: Reliable Software Releases through Build, Test, and Deployment Automation*. Boston : Addison-Wesley, 2010. ISBN 978-0-321-60191-9.
7. OPENAI. ChatGPT macOS app—Screenshot Tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://help.openai.com/en/articles/9295245-chatgpt-macos-app-screenshot-tool>
8. OPENAI. GPT-5.6: Frontier intelligence that scales with your ambition. Online. 2026. [Accessed 20 září 2026]. Available from: <https://openai.com/index/gpt-5-6/>
9. ANTHROPIC. Introducing Claude Opus 5. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.anthropic.com/news/claude-opus-5>
10. DEEPSEEK. Introducing DeepSeek-V4.1-Flash: smarter, faster, more efficient. Online. 2026. [Accessed 20 září 2026]. Available from: <https://www.deepseek.com/en/news/deepseek-v4-1-flash/>
11. MIKOLOV, Tomas, CHEN, Kai, CORRADO, Greg a DEAN, Jeffrey. Efficient Estimation of Word Representations in Vector Space. *arXiv preprint arXiv:1301.3781*. Online. 2013. Available from: <https://arxiv.org/abs/1301.3781>
12. KARPATY, Andrej. A new kind of coding I call vibe coding. Online. 2025. [Accessed 20 září 2026]. Available from: <https://twitter.com/karpathy/status/1886192184808149383>
13. WILLISON, Simon. A quote from Andrej Karpathy. Online. 2025. [Accessed 20 září 2026]. Available from: <https://simonwillison.net/2025/Feb/6/andrej-karpathy/>
14. CAMBRIDGE UNIVERSITY PRESS. AI slop. Online. 2026. [Accessed 20 září 2026]. Available from: <https://dictionary.cambridge.org/dictionary/english/ai-slop>

15. ANTHROPIC. Navigating the Claude desktop app. Online. 2026. [Accessed 20 září 2026]. Available from: <https://academy.claude.com/tutorials/navigating-the-claude-desktop-app>
16. GVISOR AUTHORS. Introduction to gVisor security. Online. 2026. [Accessed 20 září 2026]. Available from: https://gvisor.dev/docs/architecture_guide/intro/
17. AGACHE, Alexandru, DEACONESCU, Razvan, IORDACHE, Mihai, LUPU, Alexandra, RADU, Vlad, BUGA, Cezar, BALAN, Catalin a FLORESCU, Andreea. Firecracker: Lightweight Virtualization for Serverless Applications. In : *17th USENIX Symposium on Networked Systems Design and Implementation (NSDI 20)*. Online. 2020. p. 419–434. Available from: <https://www.usenix.org/conference/nsdi20/presentation/agache>
18. MOSQUEIRA-REY, Eduardo, HERNÁNDEZ-PEREIRA, Elena, ALONSO-RÍOS, David, BOBES-BASCARÁN, José a FERNÁNDEZ-LEAL, Ángel. Human-in-the-loop machine learning: a state of the art. *Artificial Intelligence Review*. Online. 2023. Vol. 56, no. 4, p. 3005–3054. Available from: <https://doi.org/10.1007/s10462-022-10246-w>
19. GONÇALVES, Pavlína Wurzel, FREGNAN, Enrico, BAUM, Tobias, SCHNEIDER, Kurt a BACCHELLI, Alberto. Do explicit review strategies improve code review performance? Towards understanding the role of cognitive load. *Empirical Software Engineering*. Online. 2022. Vol. 27, no. 4, p. 99. DOI [10.1007/s10664-022-10123-8](https://doi.org/10.1007/s10664-022-10123-8).
20. YAO, Shunyu, ZHAO, Jeffrey, YU, Dian, DU, Nan, SHAFRAN, Izhak, NARASIMHAN, Karthik a CAO, Yuan. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*. Online. 2022. Available from: <https://arxiv.org/abs/2210.03629>
21. SHINN, Noah, CASSANO, Federico, GOPINATH, Ashwin, NARASIMHAN, Karthik a YAO, Shunyu. Reflexion: Language Agents with Verbal Reinforcement Learning. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 8634–8652. Available from: <https://arxiv.org/abs/2303.11366>
22. WRIGHT, Austin, ANDREWS, Henry, HUTTON, Ben a DENNIS, Greg. JSON Schema Draft 2020-12. Online. 2022. [Accessed 20 září 2026]. Available from: <https://json-schema.org/draft/2020-12>
23. OPENAI. Introducing Structured Outputs in the API. Online. 2024. [Accessed 20 září 2026]. Available from: <https://openai.com/index/introducing-structured-outputs-in-the-api/>
24. ANTHROPIC. Code execution tool. Online. 2026. [Accessed 20 září 2026]. Available from: <https://platform.claude.com/docs/en/agents-and-tools/tool-use/code-execution-tool>
25. ANTHROPIC. Model Context Protocol documentation. Online. 2026. Available from: <https://modelcontextprotocol.io/docs/2026-07-28/getting-started/intro#explore-mcp>

26. AGENT SKILLS. Agent Skills Specification. Online. 2026. [Accessed 20 září 2026]. Available from: <https://agentskills.io/specification>
27. JIANG, Huiqiang, WU, Qianhui, LIN, Chin-Yew, YANG, Yuqing a QIU, Lili. LLMingua: Compressing Context for Accelerated Inference of Large Language Models. *arXiv preprint arXiv:2310.05736*. Online. 2023. Available from: <https://arxiv.org/abs/2310.05736>
28. LEWIS, Patrick, PEREZ, Ethan, PIKTUS, Aleksandra, PETRONI, Fabio, KARPUKHIN, Vladimir, GOYAL, Naman, KÜTTLER, Heinrich, LEWIS, Mike, YIH, Wen-tau, ROCKTÄSCHEL, Tim, RIEDEL, Sebastian a KIELA, Douwe. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Advances in Neural Information Processing Systems*. Online. 2020. Vol. 33, p. 9459–9474. Available from: <https://arxiv.org/abs/2005.11401>
29. LIU, Nelson F., LIN, Kevin, HEWITT, John, PARANJAPE, Ashwin, BEVILACQUA, Michele, PETRONI, Fabio a LIANG, Percy. Lost in the Middle: How Language Models Use Long Contexts. *Transactions of the Association for Computational Linguistics*. Online. 2024. Vol. 12, p. 157–173. Available from: <https://arxiv.org/abs/2307.03172>
30. MARIUS, Patrik. DarkFactory. Online. 2026. [Accessed 20 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
31. MARIUS, Patrik. DarkFactory: autonomous, governed software engineering pipelines. Online. 2026. [Accessed 8 září 2026]. Available from: <https://github.com/marius-patrik/DarkFactory>
32. VASWANI, Ashish, SHAZEER, Noam, PARMAR, Niki, USZKOREIT, Jakob, JONES, Llion, GOMEZ, Aidan N., KAISER, Łukasz a POLOSUKHIN, Illia. Attention Is All You Need. *arXiv preprint arXiv:1706.03762*. Online. 2017. Available from: <https://arxiv.org/abs/1706.03762>
33. Deepseek Harness. *arXiv preprint arXiv:2608.25512*. Online. 2026. Available from: <https://arxiv.org/abs/2608.25512>
34. ANTHROPIC. Prompt Engineering overview. Online. 2026. Available from: <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/overview>
35. DAO, Tri, FU, Daniel Y., ERMON, Stefano, RUDRA, Atri a RÉ, Christopher. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. *Advances in Neural Information Processing Systems*. Online. 2022. Vol. 35, p. 16344–16359. Available from: <https://arxiv.org/abs/2205.14135>
36. AINSLIE, Joshua, LEE-THORP, James, JONG, Michiel de, ZEMLYANSKIY, Yury, LEBRÓN, Federico a SANGHAI, Sumit. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints. *arXiv preprint arXiv:2305.13245*. Online. 2023. Available from: <https://arxiv.org/abs/2305.13245>
37. WOOLDRIDGE, Michael a JENNINGS, Nicholas R. Intelligent Agents: Theory and Practice. *The Knowledge Engineering Review*. 1995. Vol. 10, no. 2, p. 115–152.

38. WANG, Lei, MA, Chen, FENG, Xueyang, ZHANG, Zeyu, YANG, Hao, ZHANG, Jingsen, CHEN, Zhiyuan, TANG, Jiakai, CHEN, Xu, LIN, Yankai, ZHAO, Wayne Xin, WEI, Zhewei a WEN, Ji-Rong. A Survey on Large Language Model Based Autonomous Agents. *Frontiers of Computer Science*. Online. 2024. Vol. 18, no. 6, p. 186345. Available from: <https://arxiv.org/abs/2308.11432>
39. SCHICK, Timo, DWIVEDI-YU, Jane, DESS\I, Roberto, RAILEANU, Roberta, LOMELI, Maria, ZETTLEMOYER, Luke, CANCEDDA, Nicola a SCIALOM, Thomas. Toolformer: Language Models Can Teach Themselves to Use Tools. *Advances in Neural Information Processing Systems*. Online. 2023. Vol. 36, p. 68539–68551. Available from: <https://arxiv.org/abs/2302.04761>
40. SENNRICH, Rico, HADDOW, Barry a BIRCH, Alexandra. Neural Machine Translation of Rare Words with Subword Units. *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Online. 2016. P. 1715–1725. Available from: <https://aclanthology.org/P16-1162/>
41. WU, Qingyun, BANSAL, Gagan, ZHANG, Jieyu, WU, Yiran, LI, Beibin, ZHU, Erkang, JIANG, Li, ZHANG, Xiaoyun, ZHANG, Chi, LIU, Jue, AWADALLAH, Ahmed Hassan, WHITE, Ryen W., BURGER, Doug a WANG, Chi. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155*. Online. 2023. Available from: <https://arxiv.org/abs/2308.08155>
42. SOMMERVILLE, Ian. *Software Engineering*. 10. Pearson, 2016. ISBN 978-0-13-394303-0.
43. ANTHROPIC. Building effective agents. Online. 2024. Available from: <https://www.anthropic.com/research/building-effective-agents>
44. MERKEL, Dirk. Docker: lightweight linux containers for consistent development and deployment. *Linux Journal*. 2014. Vol. 2014, no. 239, p. 2.
45. KINSMAN, Timothy, WESSEL, Mairieli, GEROSA, Marco A. a TREUDE, Christoph. How do software developers use GitHub Actions to automate their workflows?. In : *Proceedings of the 18th International Conference on Mining Software Repositories*. 2021. p. 420–431.
46. DABBISH, Laura, STUART, Colleen, TSAY, Jason a HERBSLEB, Jim. Social coding in GitHub: transparency and collaboration in an open software repository. In : *Proceedings of the ACM 2012 conference on Computer Supported Cooperative Work*. 2012. p. 1277–1286.
47. FOWLER, Martin. Vibe Coding. Online. 2026. [Accessed 20 září 2026]. Available from: <https://martinfowler.com/bliki/VibeCoding.html>